


# DATA STRUCTURES AND ALGORITHMS

## 23CSE203

## LAB MANUAL

Submitted by

| | |
|---|---|
| NAME | S.Sai Sahitya |
| ROLL NO | AV.SC.U4CSE25241 |
| YEAR/SEM/SECTION | 2nd , 3rd Sem, CSE-C |
| CREDITS | 05 |

Submitted to

| | |
|---|---|
| NAME | Dr. Gayatri Parasa |
| DESIGNATION | Asst. Proffessor(Sr.Gr) |
| DEPARTMENT | School Of Computing |

| S.No | Title | Page No |
|---|---|---|
| 1. | WEEK-1:<br><br>**1(A).** A rocket launch system displays a countdown before launch. Instead of using a loop, implement the countdown using recursion. Write a recursive python program to display a countdown from n to 1, followed by "Launch!".<br><br>**1(B).** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate \(\text{P}^{n}\)(power) where: P= principal growth factor, n= number of years.<br><br>**1(C).** A HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.<br><br>**1(D).** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.<br><br>**1(E).** Write a recursive python program to print the first n terms of the Fibonacci series. | 11-16 |
| **2** | WEEK 2:<br>**2(A).** Write a python program to implement linear search.<br>**2(B).** Write a python program to implement binary search with and without sorted array as an input. | 17-20 |
| **3** | WEEK-2<br>**3(A).** Write a python program to implement bubble sort.<br>**3(B).** Write a python program to implement insertion sort.<br>**3(C).** Write a python program to implement selection sort. | 21-24 |
| **4.** | WEEK-4**:**<br>**4(A).** Write a python program to implement quick sort.<br>**4(B).** Write a python program to implement merge sort. | 25-27 |
| **5.** | WEEK-5**:**<br>**5(A).** Write a python program to implement a Singly Linked List with the below operations.<br><br>1. Create Linked List | 28-29 |

| | | |
|---|---|---|
| | 2. Insert Beginning<br>3. Insert End<br>4. Insert at Index<br>5. Delete by Value<br>6. Delete First Node<br>7. Delete Last Node<br>8. Count number of nodes<br>9. Sort the data of SLL<br>10. Search data in SLL<br>11. Display or Traverse<br>12. Exit | |
| **6.** | **WEEK-6:**<br>Write a python program to implement a Double Linked List with the below operations.<br>1. Create Linked List<br>2. Insert Beginning<br>3. Insert End<br>4. Insert at Index<br>5. Delete by Value<br>6. Delete First Node<br>7. Delete Last Node<br>8. Count number of nodes<br>9. Display or Traverse<br>10. Exit | 30-33 |
| **7.** | **WEEK-7:**<br>**7(A).** Write a python program to implement stack operations such as push, pop, peek, display using an array.<br>**7(B).** Write a python program to implement stack operations such as push, pop, peek, display using a linked list. | 34-35 |
| **8.** | **WEEK-8:**<br>**8(A).** Write a python program to implement Queue and its | 36-37 |

| | | |
|---|---|---|
| | operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures.<br>**8(B).** Write a python program to implement Circular Queue and its operations such as enqueue, dequeue, peek, display with a menu driven approach. Implement using array, and linked list data structures. | |
| **9.** | **WEEK-9:**<br>Write a python program to implement a Binary Search Tree (BST) and perform the following operations.<br>1. Insertion<br>2. Deletion<br>3. Tree Traversals – in-order, pre-order and post-order<br>4. Count Leaf Nodes<br>5. Count Internal Nodes<br>6. Height of the Tree | 38-39 |
| **10.** | Write a menu driven python program to implement an AVL Tree and perform the following operations.<br>1. Insertion<br>2. Deletion<br>3. Tree Traversals<br>4. Count Leaf Nodes<br>5. Count Internal Nodes<br>6. Height of the Tree | 40-41 |
| **11** | **WEEK-11:**<br>Write a menu-driven Python program to implement a Binary Heap (Max Heap and Min Heap) and perform the following operations:<br>1. Insertion of an element into the Heap.<br>2. Deletion of the root element from the Heap.<br>3. Display the array representation of the Heap.<br>4. Exit the program | 42-43 |

| | | |
|---|---|---|
| **12.** | WEEK-12:<br>Write a program to implement the following three collision resolution techniques in a Hash Table:<br>1. Separate Chaining<br>2. Linear Probing<br>3. Quadratic Probing | 44-47 |

# Week-1

1) **Aim**: A rocket launch system displays a countdown before launch. Instead of using a loop implement the countdown using recursion. Write a recursive program to display a countdown from n to 1 followed by "Lunch!"

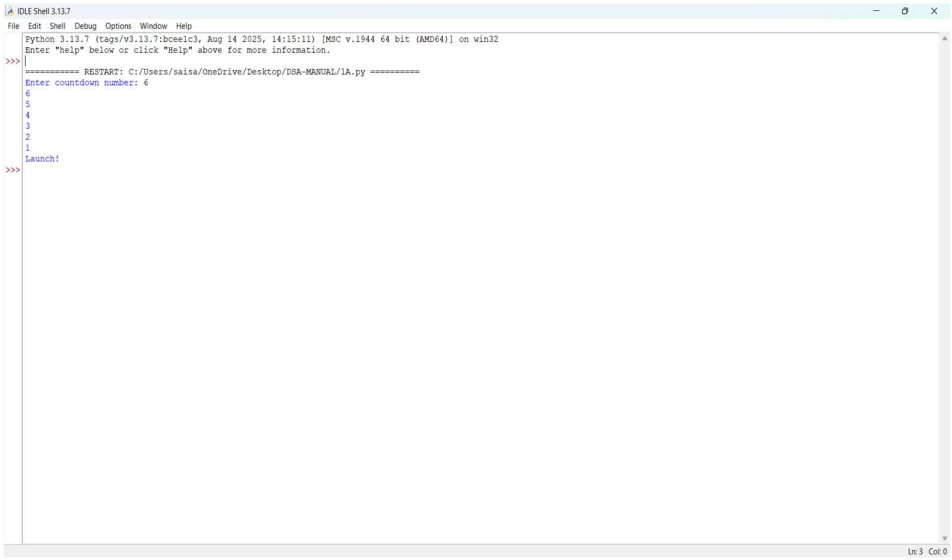
**CODE:**

```
File  Edit  Format  Run  Options  Window  Help
def countdown(n):
    if n == 0:
        print("Launch!")
        return

    print(n)
    countdown(n - 1)

n = int(input("Enter countdown number: "))
countdown(n)
```

**OUTPUT:**




b) **AIM:** Banks calculate growth of the investment using compound interset. The formula requires calculating powers which can be implemented recursively. Write a recursive program calculate (power), where p = principal growth factor, n = no. of years.

**CODE:**

```
File  Edit  Format  Run  Options  Window  Help
def power(p, n):
    if n == 0:
        return 1

    return p * power(p, n - 1)

p = int(input("Enter principal growth factor: "))
n = int(input("Enter number of years: "))

print("Power =", power(p, n))
```

**CODE:**

```
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/1A.py ==========
Enter principal growth factor: 6
Enter number of years: 3
Power = 216
```

c) An HR department stores employee IDs in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

**CODE:**

```
File  Edit  Format  Run  Options  Window  Help
def search(list, key, index):
    if index == len(list):
        return False

    if list[index] == key:
        return True

    return search(list, key, index + 1)

emp_ids = [101, 102, 103, 104, 105]

key = int(input("Enter employee ID to search: "))

if search(emp_ids, key, 0):
    print("Employee ID found")
else:
    print("Employee ID not found")
```

**OUTPUT:**

```
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/1A.py =
Enter employee ID to search: 103
Employee ID found
```

d) In a logistics company, the number of possible ways to arrange (n) different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

**CODE:**

```
File  Edit  Format  Run  Options  Window  Help
def factorial(n):
    if n == 0 or n == 1:
        return 1

    return n * factorial(n - 1)

num = int(input("Enter a number: "))

print("Factorial =", factorial(num))
```

**OUTPUT:**

```
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/1A.py
Enter a number: 3
Factorial = 6
```

e) Write a recursive Python program to print the first \(n\) terms of the Fibonacci series.

**CODE:**

```
File  Edit  Format  Run  Options  Window  Help
def fibonacci(n):
    if n == 0:
        return 0
    elif n == 1:
        return 1

    return fibonacci(n - 1) + fibonacci(n - 2)

terms = int(input("Enter number of terms: "))

print("Fibonacci Series:")

for i in range(terms):
    print(fibonacci(i), end=" ")
```

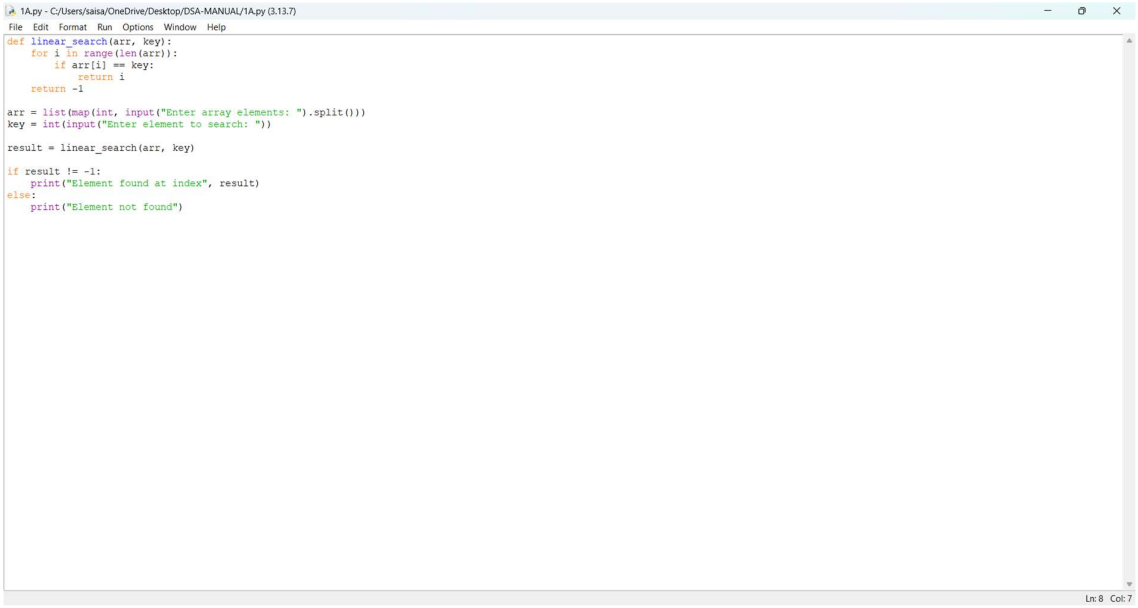
**OUTPUT:**

```
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/1A.py =====
Enter number of terms: 4
Fibonacci Series:
0 1 1 2
```

# Week-2

2)Linear Search:

**CODE:**

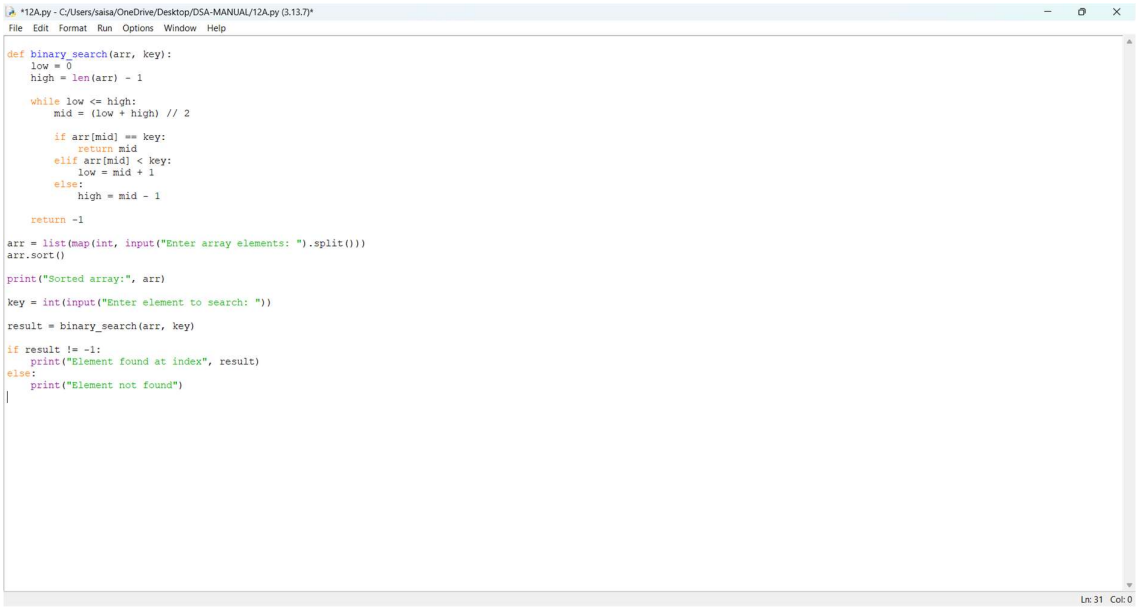



**OUTPUT:**

```
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/1A.py ==========
Enter array elements: 4
Enter element to search: 3
Element not found
>>>
```

b) Binary Search:

**CODE:**



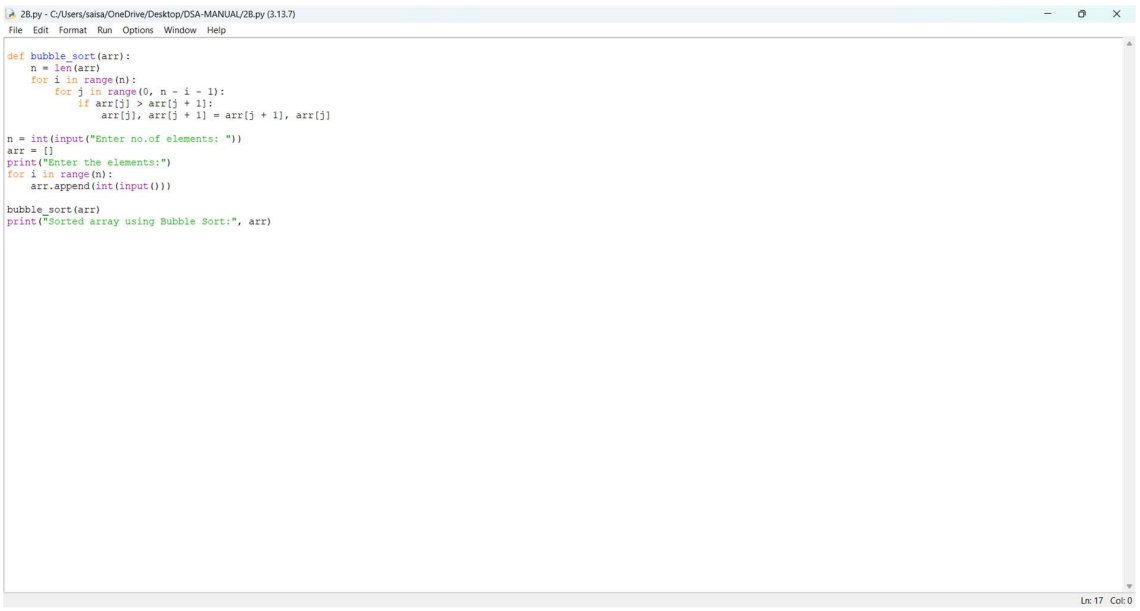

# OUTPUT:

```
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/1A.py ==========
Enter array elements: 4
Sorted array: [4]
Enter element to search: 2
Element not found
>>
```

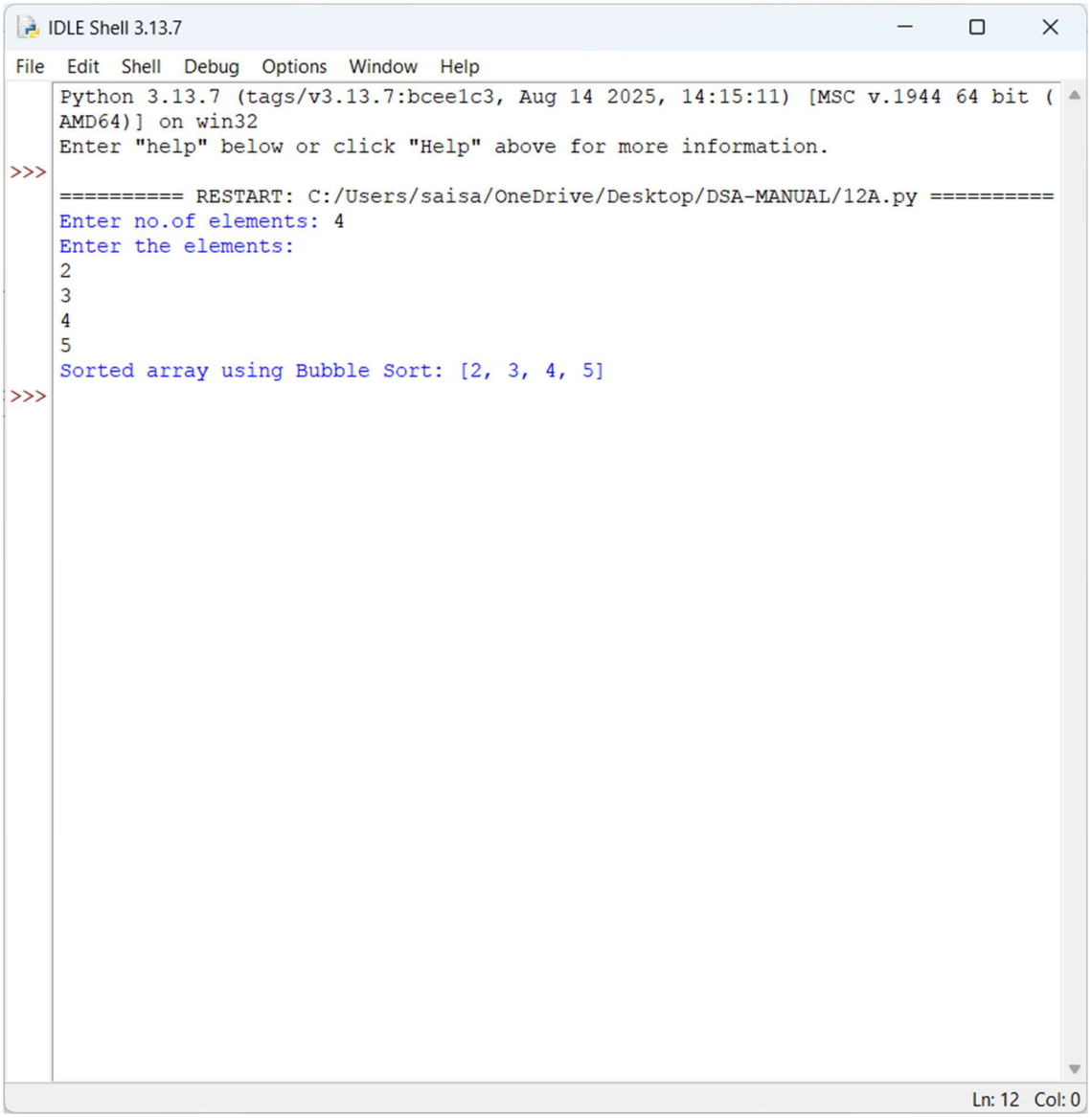
# Week-3

3a) Bubble Sort:

## CODE:

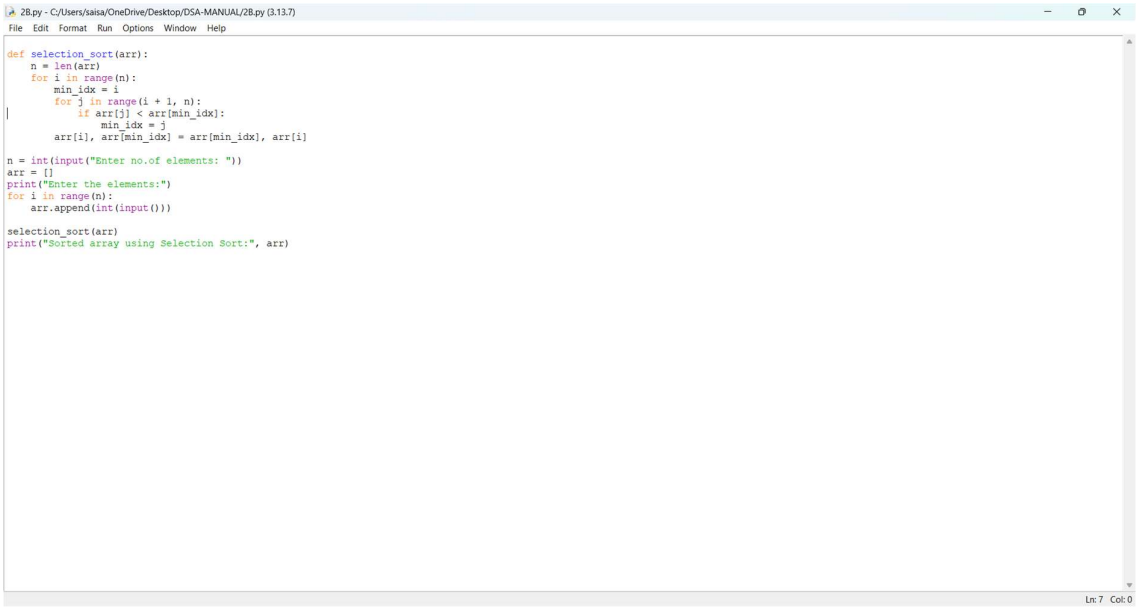



## OUTPUT:

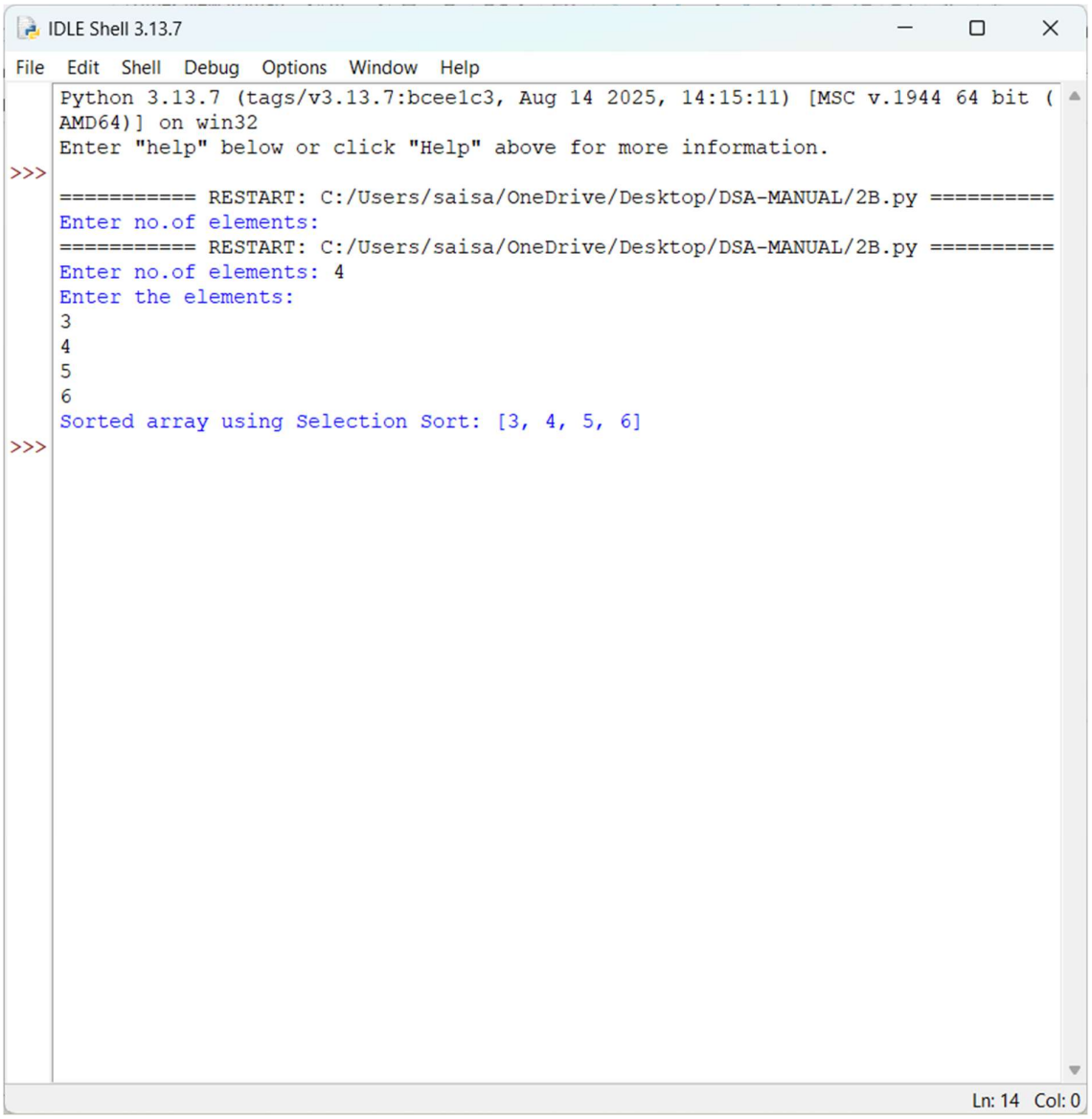



3b) Selection Sort:

## CODE:

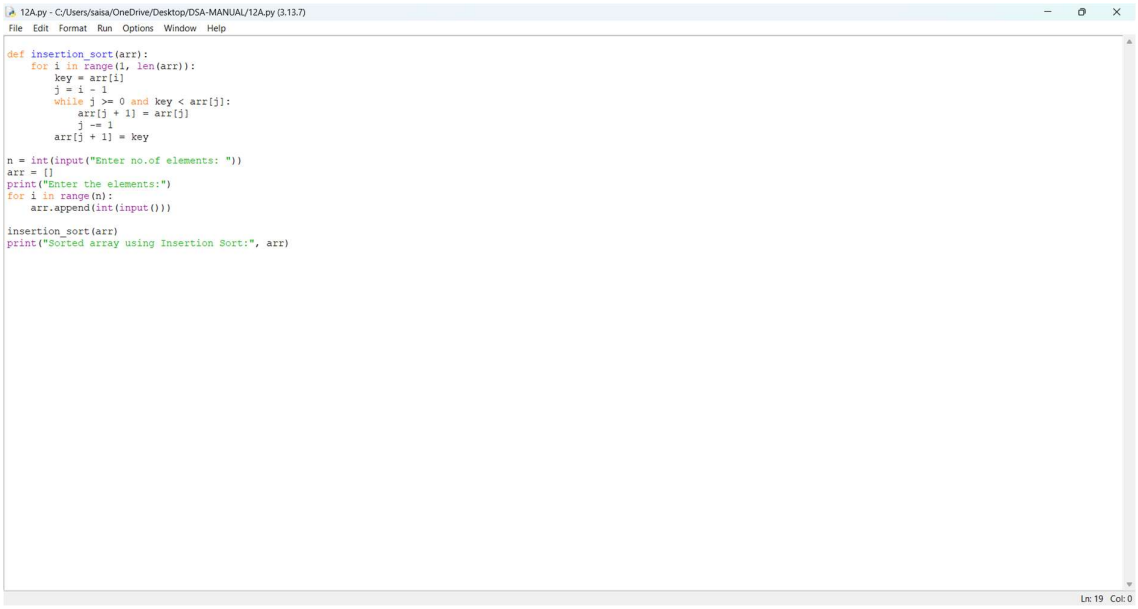

2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)
File Edit Format Run Options Window Help
def selection_sort(arr):
n = len(arr)
for i in range(n):
min_idx = i
for j in range(i + 1, n):
if arr[j] < arr[min_idx]:
min_idx = j
arr[i], arr[min_idx] = arr[min_idx], arr[i]
n = int(input("Enter no.of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
arr.append(int(input()))
selection_sort(arr)
print("Sorted array using Selection Sort:", arr)
Ln: 7 Col: 0

## OUTPUT:




3c) Insertion Sort:

**CODE:**


12A.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/12A.py (3.13.7)
File Edit Format Run Options Window Help
def insertion_sort(arr):
for i in range(1, len(arr)):
key = arr[i]
j = i - 1
while j >= 0 and key < arr[j]:
arr[j + 1] = arr[j]
j -= 1
arr[j + 1] = key
n = int(input("Enter no.of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
arr.append(int(input()))
insertion_sort(arr)
print("Sorted array using Insertion Sort:", arr)
Ln: 19 Col: 0

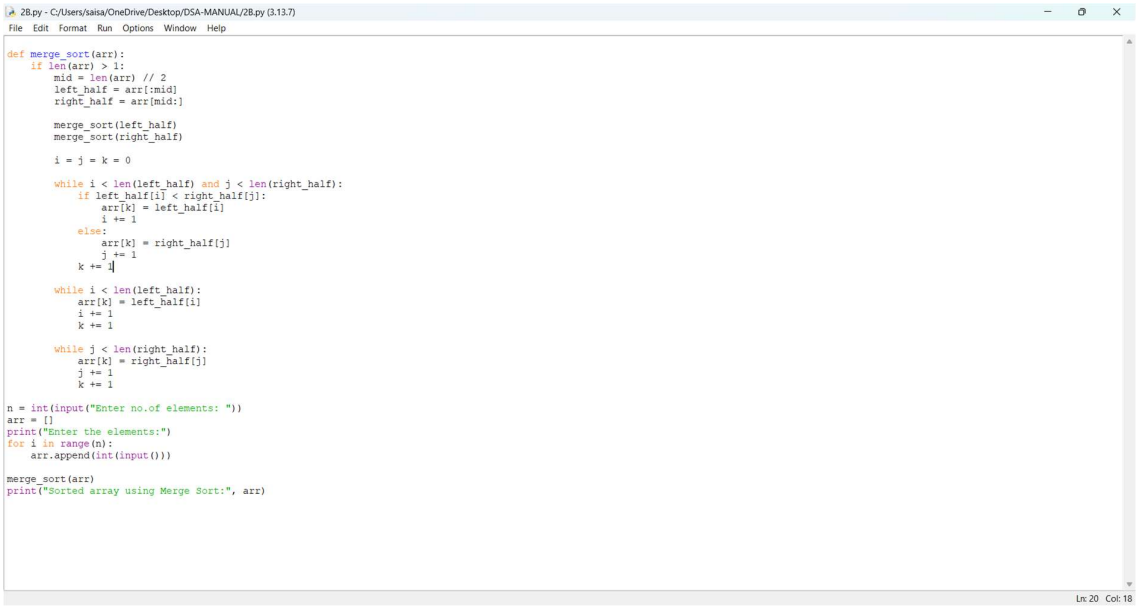
## OUTPUT:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/12A.py ==========
Enter no.of elements: 4
Enter the elements:
2
3
4
5
Sorted array using Insertion Sort: [2, 3, 4, 5]
>>>
Ln: 12 Col: 0

# Week-4

4a) Merge Sort:

## CODE:

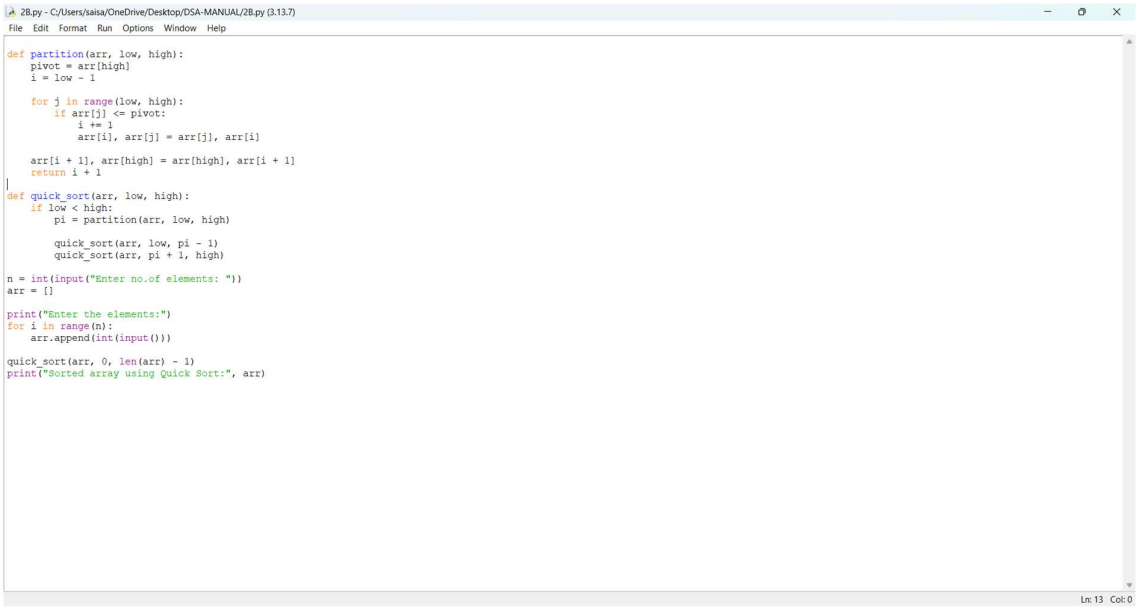



## OUTPUT:




4b) Quick Sort:

**CODE:**


2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)
File Edit Format Run Options Window Help
def partition(arr, low, high):
pivot = arr[high]
i = low - 1
for j in range(low, high):
if arr[j] <= pivot:
i += 1
arr[i], arr[j] = arr[j], arr[i]
arr[i + 1], arr[high] = arr[high], arr[i + 1]
return i + 1
def quick_sort(arr, low, high):
if low < high:
pi = partition(arr, low, high)
quick_sort(arr, low, pi - 1)
quick_sort(arr, pi + 1, high)
n = int(input("Enter no.of elements: "))
arr = []
print("Enter the elements:")
for i in range(n):
arr.append(int(input()))
quick_sort(arr, 0, len(arr) - 1)
print("Sorted array using Quick Sort:", arr)
Ln: 13 Col: 0

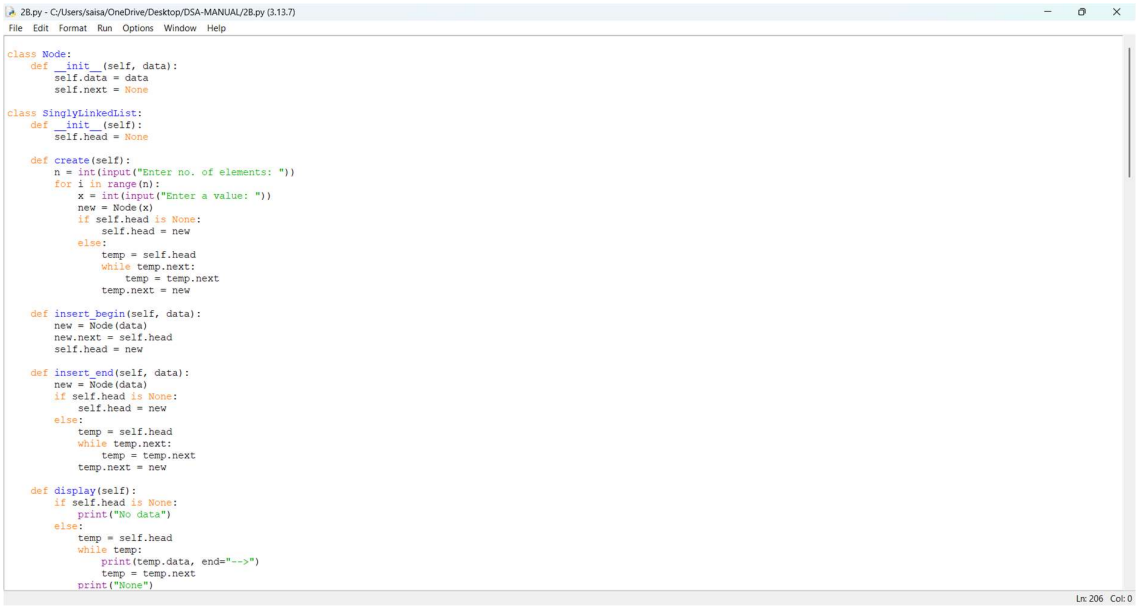
## OUTPUT:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py ==========
Enter no.of elements: 4
Enter the elements:
3
2
6
7
Sorted array using Quick Sort: [2, 3, 6, 7]
>>>
Ln: 12 Col: 0

# Week-5

5)Singly Linked List:

## CODE:



```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

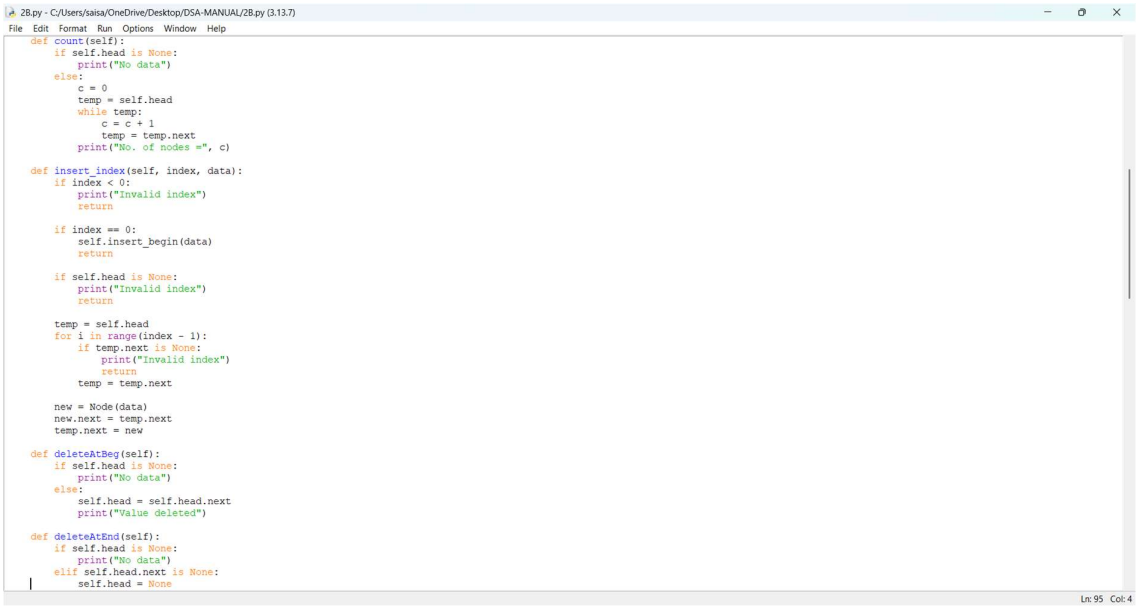
class SinglyLinkedList:
    def __init__(self):
        self.head = None

    def create(self):
        n = int(input("Enter no. of elements: "))
        for i in range(n):
            x = int(input("Enter a value: "))
            new = Node(x)
            if self.head is None:
                self.head = new
            else:
                temp = self.head
                while temp.next:
                    temp = temp.next
                temp.next = new

    def insert_begin(self, data):
        new = Node(data)
        new.next = self.head
        self.head = new

    def insert_end(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
        else:
            temp = self.head
            while temp.next:
                temp = temp.next
            temp.next = new

    def display(self):
        if self.head is None:
            print("No data")
        else:
            temp = self.head
            while temp:
                print(temp.data, end="-->")
                temp = temp.next
            print("None")
```



```
    def count(self):
        if self.head is None:
            print("No data")
        else:
            c = 0
            temp = self.head
            while temp:
                c = c + 1
                temp = temp.next
            print("No. of nodes =", c)

    def insert_index(self, index, data):
        if index < 0:
            print("Invalid index")
            return

        if index == 0:
            self.insert_begin(data)
            return

        if self.head is None:
            print("Invalid index")
            return

        temp = self.head
        for i in range(index - 1):
            if temp.next is None:
                print("Invalid index")
                return
            temp = temp.next

        new = Node(data)
        new.next = temp.next
        temp.next = new

    def deleteAtBeg(self):
        if self.head is None:
            print("No data")
        else:
            self.head = self.head.next
            print("Value deleted")

    def deleteAtEnd(self):
        if self.head is None:
            print("No data")
        elif self.head.next is None:
            self.head = None
```

2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)

File Edit Format Run Options Window Help

```
        else:
            temp = self.head
            while temp.next.next:
                temp = temp.next
            temp.next = None
            print("Value deleted")

    def delete(self, value):
        if self.head is None:
            print("No data")
            return

        # If value is in first node
        if self.head.data == value:
            self.head = self.head.next
            print("Value deleted")
            return

        temp = self.head

        while temp.next and temp.next.data != value:
            temp = temp.next

        if temp.next is None:
            print("Value not present")
        else:
            temp.next = temp.next.next
            print("Value deleted")

# Main program
s = SinglyLinkedList()

s.create()

print("\nLinked List:")
s.display()

print("\nNumber of nodes:")
s.count()

print("\nAfter inserting at beginning:")
s.insert_begin(5)
s.display()

print("\nAfter inserting at end:")
s.insert_end(100)
s.display()
```

Ln: 143 Col: 11

2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)

File Edit Format Run Options Window Help

```
while True:
    print("\n----- SINGLY LINKED LIST -----")
    print("1. Create")
    print("2. Display")
    print("3. Count")
    print("4. Insert at Beginning")
    print("5. Insert at End")
    print("6. Insert at Index")
    print("7. Delete at Beginning")
    print("8. Delete at End")
    print("9. Delete by Value")
    print("10. Exit")

    choice = int(input("Enter your choice:"))

    if choice == 1:
        s.create()

    elif choice == 2:
        s.display()

    elif choice == 3:
        s.count()

    elif choice == 4:
        data = int(input("Enter value: "))
        s.insert_begin(data)

    elif choice == 5:
        data = int(input("Enter value: "))
        s.insert_end(data)

    elif choice == 6:
        index = int(input("Enter index: "))
        data = int(input("Enter value: "))
        s.insert_index(index, data)

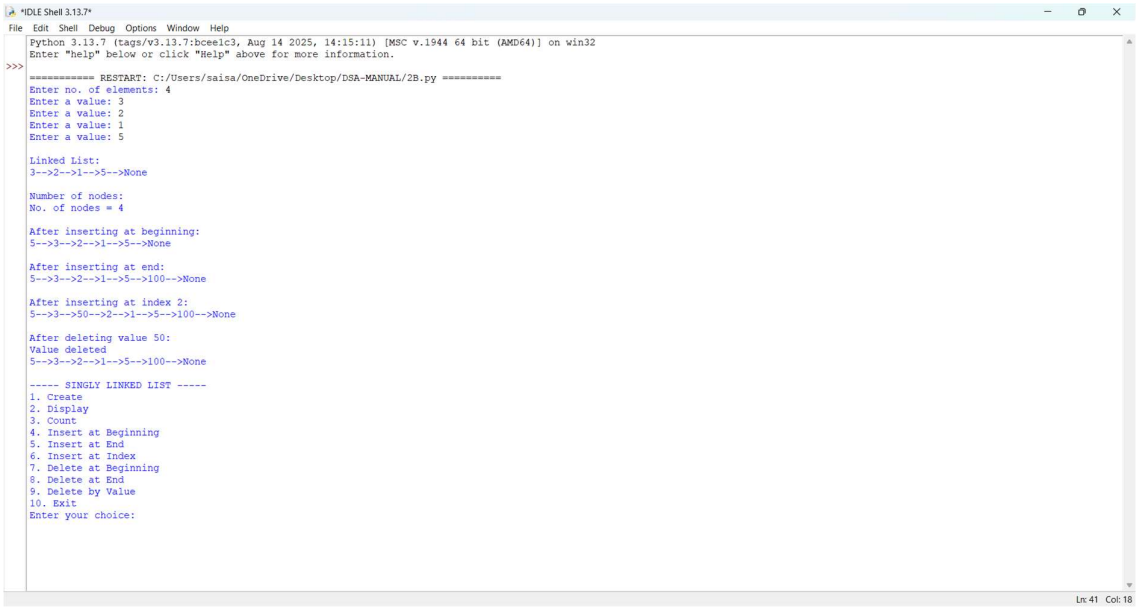
    elif choice == 7:
        s.deleteAtBeg()

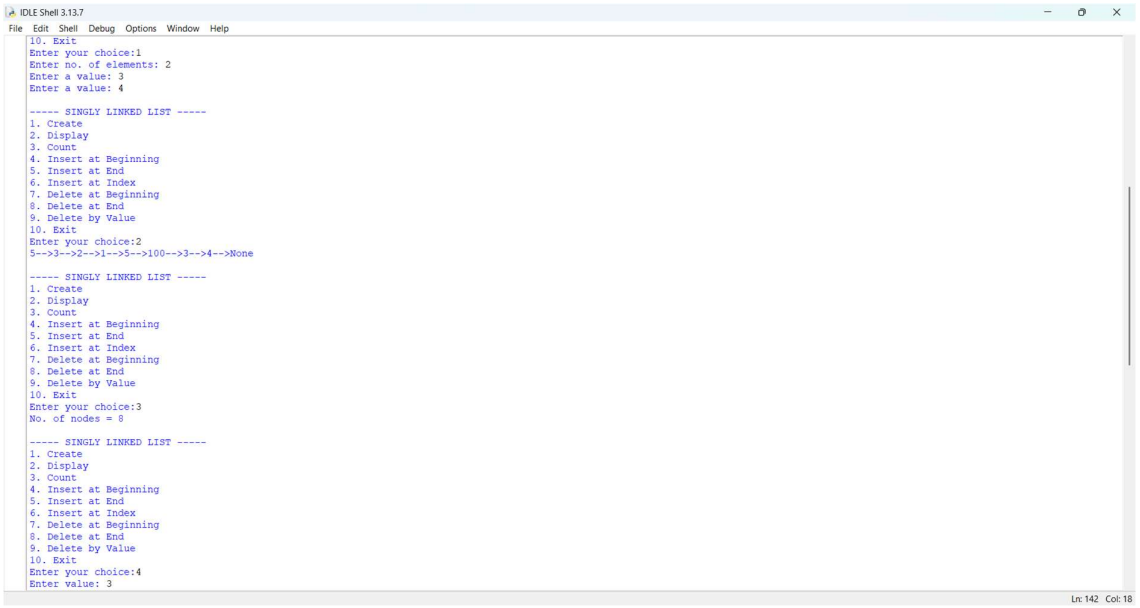
    elif choice == 8:
        s.deleteAtEnd()

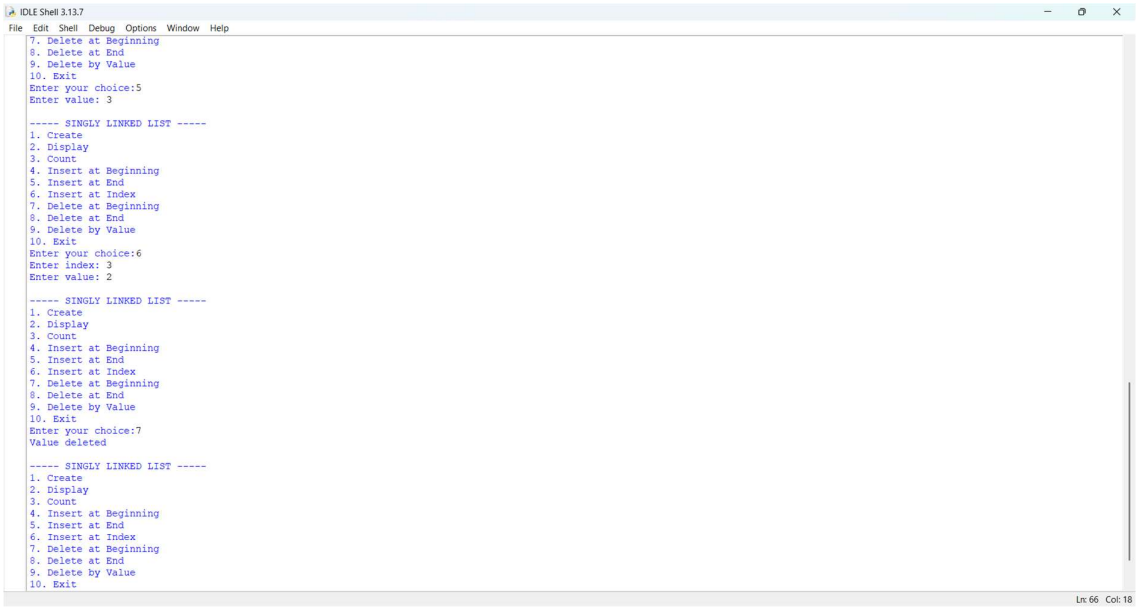
    elif choice == 9:
        value = int(input("Enter value to delete: "))
        s.delete(value)
```

Ln: 142 Col: 17

## OUTPUT:


*IDLE Shell 3.13.7*
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py ==========
Enter no. of elements: 4
Enter a value: 3
Enter a value: 2
Enter a value: 1
Enter a value: 5
Linked List:
3-->2-->1-->5-->None
Number of nodes:
No. of nodes = 4
After inserting at beginning:
5-->3-->2-->1-->5-->None
After inserting at end:
5-->3-->2-->1-->5-->100-->None
After inserting at index 2:
5-->3-->50-->2-->1-->5-->100-->None
After deleting value 50:
Value deleted
5-->3-->2-->1-->5-->100-->None
----- SINGLY LINKED LIST -----
1. Create
2. Display
3. Count
4. Insert at Beginning
5. Insert at End
6. Insert at Index
7. Delete at Beginning
8. Delete at End
9. Delete by Value
10. Exit
Enter your choice:
Ln: 41 Col: 18

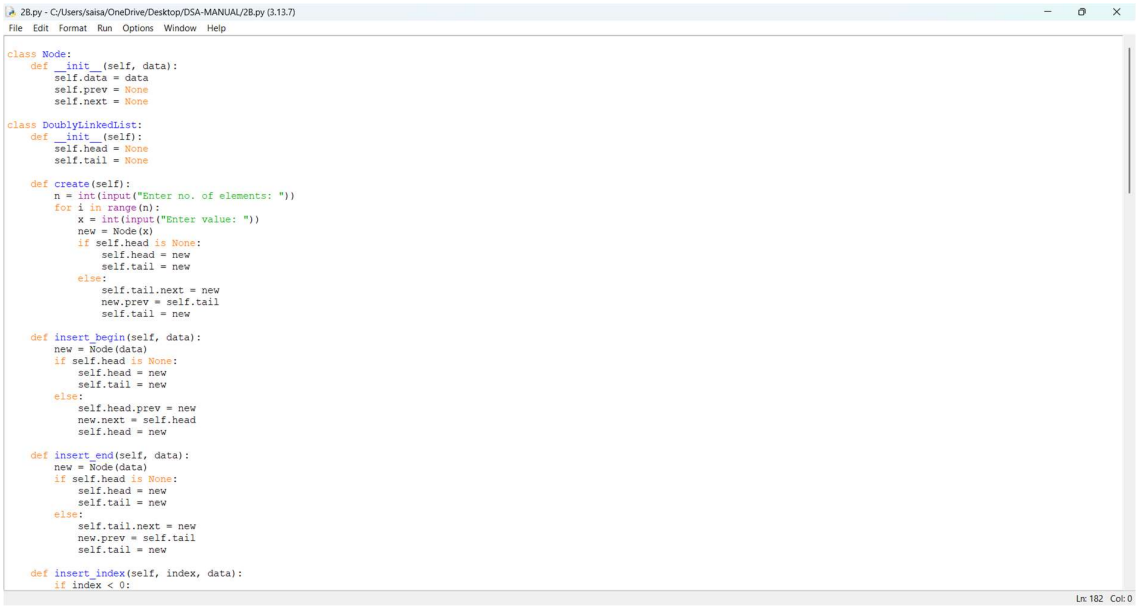

IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
10. Exit
Enter your choice:1
Enter no. of elements: 2
Enter a value: 3
Enter a value: 4
----- SINGLY LINKED LIST -----
1. Create
2. Display
3. Count
4. Insert at Beginning
5. Insert at End
6. Insert at Index
7. Delete at Beginning
8. Delete at End
9. Delete by Value
10. Exit
Enter your choice:2
5-->3-->2-->1-->5-->100-->3-->4-->None
----- SINGLY LINKED LIST -----
1. Create
2. Display
3. Count
4. Insert at Beginning
5. Insert at End
6. Insert at Index
7. Delete at Beginning
8. Delete at End
9. Delete by Value
10. Exit
Enter your choice:3
No. of nodes = 8
----- SINGLY LINKED LIST -----
1. Create
2. Display
3. Count
4. Insert at Beginning
5. Insert at End
6. Insert at Index
7. Delete at Beginning
8. Delete at End
9. Delete by Value
10. Exit
Enter your choice:4
Enter value: 3
Ln: 142 Col: 18


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
7. Delete at Beginning
8. Delete at End
9. Delete by Value
10. Exit
Enter your choice:5
Enter value: 3
----- SINGLY LINKED LIST -----
1. Create
2. Display
3. Count
4. Insert at Beginning
5. Insert at End
6. Insert at Index
7. Delete at Beginning
8. Delete at End
9. Delete by Value
10. Exit
Enter your choice:6
Enter index: 3
Enter value: 2
----- SINGLY LINKED LIST -----
1. Create
2. Display
3. Count
4. Insert at Beginning
5. Insert at End
6. Insert at Index
7. Delete at Beginning
8. Delete at End
9. Delete by Value
10. Exit
Enter your choice:7
Value deleted
----- SINGLY LINKED LIST -----
1. Create
2. Display
3. Count
4. Insert at Beginning
5. Insert at End
6. Insert at Index
7. Delete at Beginning
8. Delete at End
9. Delete by Value
10. Exit
Ln: 66 Col: 18

# Week-6

6)DOUBLY LINKED LIST:

## CODE:



```
2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help

class Node:
    def __init__(self, data):
        self.data = data
        self.prev = None
        self.next = None

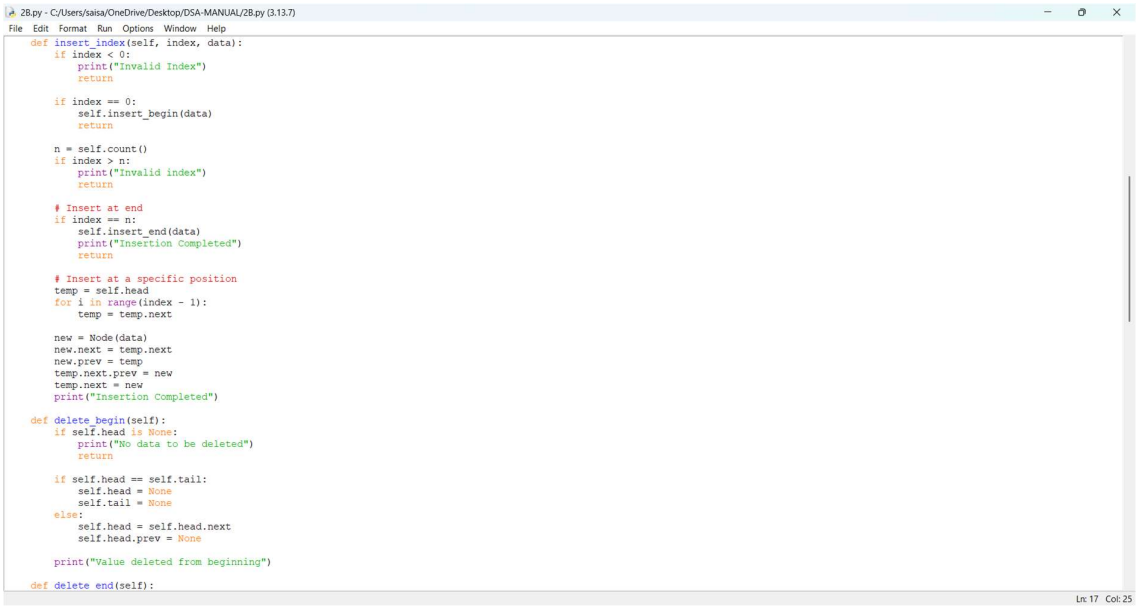
class DoublyLinkedList:
    def __init__(self):
        self.head = None
        self.tail = None

    def create(self):
        n = int(input("Enter no. of elements: "))
        for i in range(n):
            x = int(input("Enter value: "))
            new = Node(x)
            if self.head is None:
                self.head = new
                self.tail = new
            else:
                self.tail.next = new
                new.prev = self.tail
                self.tail = new

    def insert_begin(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
            self.tail = new
        else:
            self.head.prev = new
            new.next = self.head
            self.head = new

    def insert_end(self, data):
        new = Node(data)
        if self.head is None:
            self.head = new
            self.tail = new
        else:
            self.tail.next = new
            new.prev = self.tail
            self.tail = new

    def insert_index(self, index, data):
        if index < 0:
Ln: 182  Col: 0
```



```
2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help
    def insert_index(self, index, data):
        if index < 0:
            print("Invalid Index")
            return

        if index == 0:
            self.insert_begin(data)
            return

        n = self.count()
        if index > n:
            print("Invalid index")
            return

        # Insert at end
        if index == n:
            self.insert_end(data)
            print("Insertion Completed")
            return

        # Insert at a specific position
        temp = self.head
        for i in range(index - 1):
            temp = temp.next

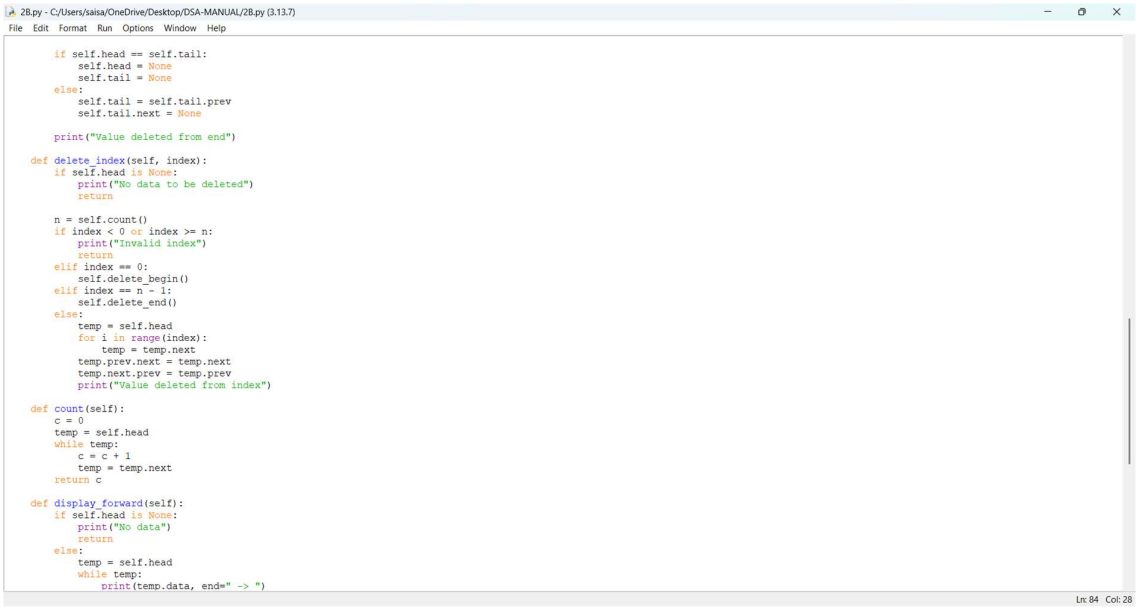
        new = Node(data)
        new.next = temp.next
        new.prev = temp
        temp.next.prev = new
        temp.next = new
        print("Insertion Completed")

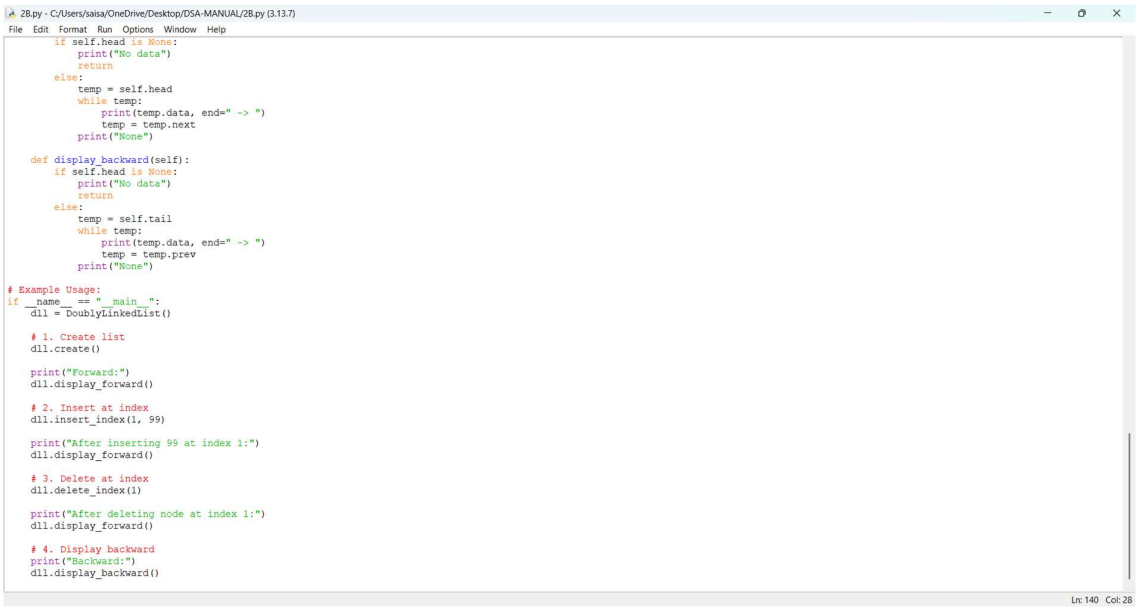
    def delete_begin(self):
        if self.head is None:
            print("No data to be deleted")
            return

        if self.head == self.tail:
            self.head = None
            self.tail = None
        else:
            self.head = self.head.next
            self.head.prev = None

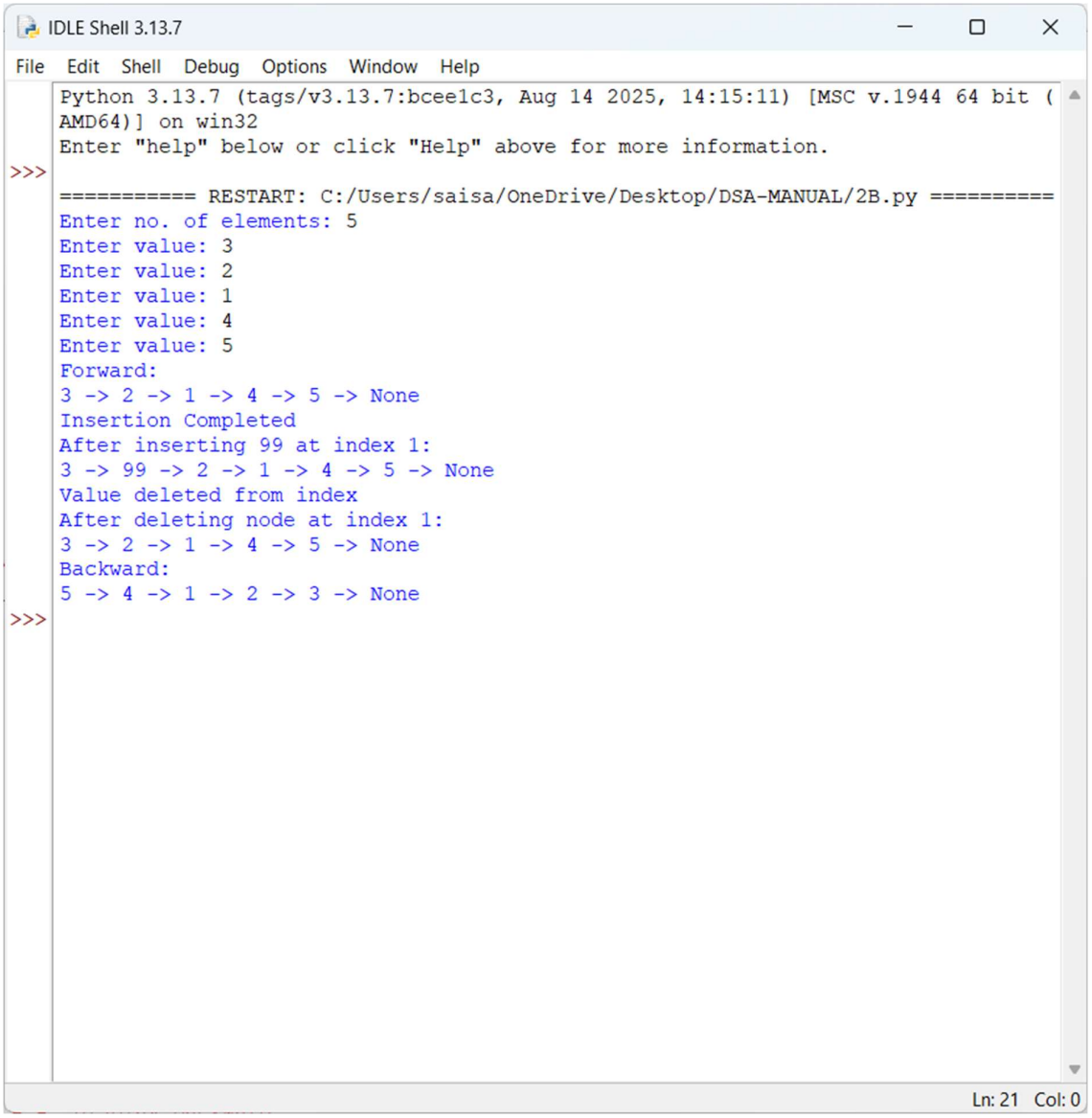
        print("Value deleted from beginning")

    def delete_end(self):
Ln: 17  Col: 25
```


2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)
File Edit Format Run Options Window Help
if self.head == self.tail:
self.head = None
self.tail = None
else:
self.tail = self.tail.prev
self.tail.next = None
print("Value deleted from end")
def delete_index(self, index):
if self.head is None:
print("No data to be deleted")
return
n = self.count()
if index < 0 or index >= n:
print("Invalid index")
return
elif index == 0:
self.delete_begin()
elif index == n - 1:
self.delete_end()
else:
temp = self.head
for i in range(index):
temp = temp.next
temp.prev.next = temp.next
temp.next.prev = temp.prev
print("Value deleted from index")
def count(self):
c = 0
temp = self.head
while temp:
c = c + 1
temp = temp.next
return c
def display_forward(self):
if self.head is None:
print("No data")
return
else:
temp = self.head
while temp:
print(temp.data, end=" -> ")
Ln: 84 Col: 28


2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)
File Edit Format Run Options Window Help
if self.head is None:
print("No data")
return
else:
temp = self.head
while temp:
print(temp.data, end=" -> ")
temp = temp.next
print("None")
def display_backward(self):
if self.head is None:
print("No data")
return
else:
temp = self.tail
while temp:
print(temp.data, end=" -> ")
temp = temp.prev
print("None")
# Example Usage:
if __name__ == "__main__":
dll = DoublyLinkedList()
# 1. Create list
dll.create()
print("Forward:")
dll.display_forward()
# 2. Insert at index
dll.insert_index(1, 99)
print("After inserting 99 at index 1:")
dll.display_forward()
# 3. Delete at index
dll.delete_index(1)
print("After deleting node at index 1:")
dll.display_forward()
# 4. Display backward
print("Backward:")
dll.display_backward()
Ln: 140 Col: 28

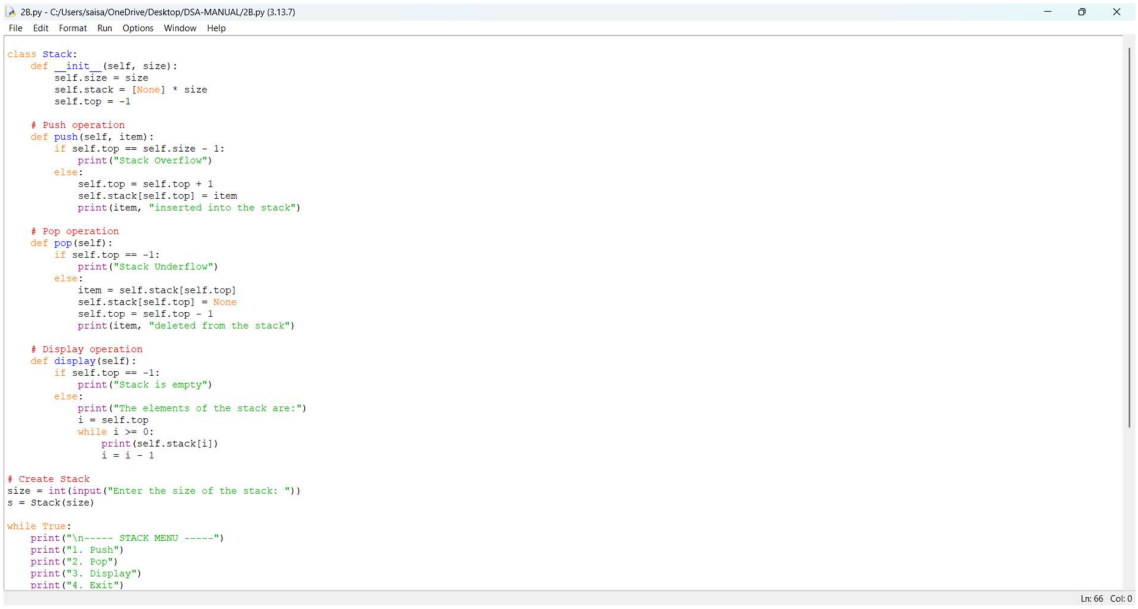
## OUTPUT:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py ==========
Enter no. of elements: 5
Enter value: 3
Enter value: 2
Enter value: 1
Enter value: 4
Enter value: 5
Forward:
3 -> 2 -> 1 -> 4 -> 5 -> None
Insertion Completed
After inserting 99 at index 1:
3 -> 99 -> 2 -> 1 -> 4 -> 5 -> None
Value deleted from index
After deleting node at index 1:
3 -> 2 -> 1 -> 4 -> 5 -> None
Backward:
5 -> 4 -> 1 -> 2 -> 3 -> None
>>>
Ln: 21 Col: 0

# Week-7

7)Stack Operations:

## CODE:



```
class Stack:
    def __init__(self, size):
        self.size = size
        self.stack = [None] * size
        self.top = -1

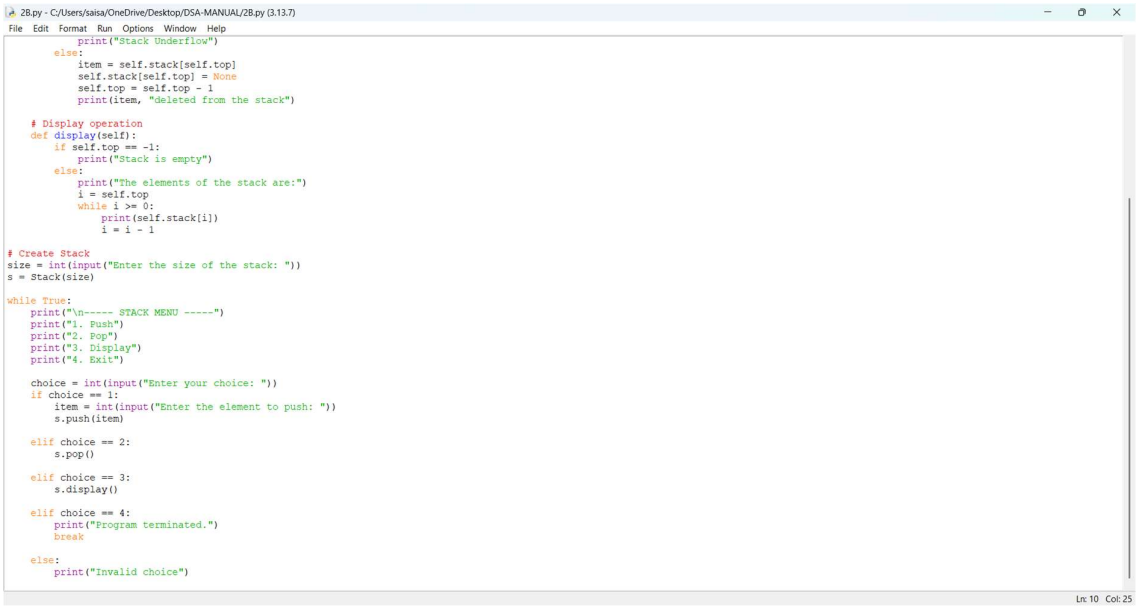
    # Push operation
    def push(self, item):
        if self.top == self.size - 1:
            print("Stack Overflow")
        else:
            self.top = self.top + 1
            self.stack[self.top] = item
            print(item, "inserted into the stack")

    # Pop operation
    def pop(self):
        if self.top == -1:
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top = self.top - 1
            print(item, "deleted from the stack")

    # Display operation
    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("The elements of the stack are:")
            i = self.top
            while i >= 0:
                print(self.stack[i])
                i = i - 1

# Create Stack
size = int(input("Enter the size of the stack: "))
s = Stack(size)

while True:
    print("\n----- STACK MENU -----")
    print("1. Push")
    print("2. Pop")
    print("3. Display")
    print("4. Exit")
```



```
            print("Stack Underflow")
        else:
            item = self.stack[self.top]
            self.stack[self.top] = None
            self.top = self.top - 1
            print(item, "deleted from the stack")

    # Display operation
    def display(self):
        if self.top == -1:
            print("Stack is empty")
        else:
            print("The elements of the stack are:")
            i = self.top
            while i >= 0:
                print(self.stack[i])
                i = i - 1

# Create Stack
size = int(input("Enter the size of the stack: "))
s = Stack(size)

while True:
    print("\n----- STACK MENU -----")
    print("1. Push")
    print("2. Pop")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))
    if choice == 1:
        item = int(input("Enter the element to push: "))
        s.push(item)

    elif choice == 2:
        s.pop()

    elif choice == 3:
        s.display()

    elif choice == 4:
        print("Program terminated.")
        break

    else:
        print("Invalid choice")
```

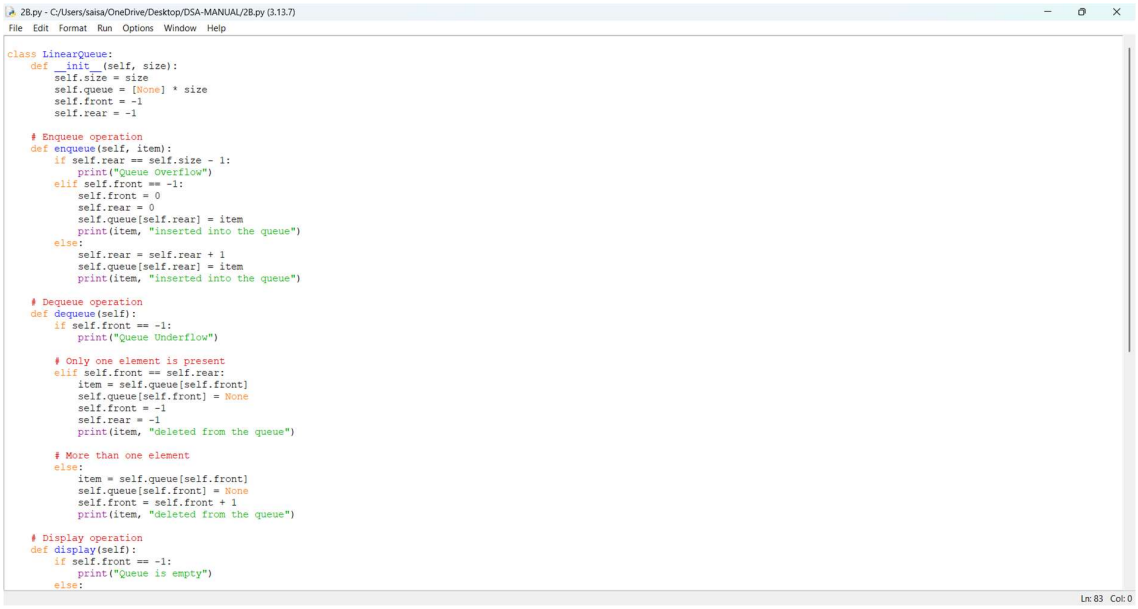
## OUTPUT:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py ==========
Enter the size of the stack: 4
----- STACK MENU -----
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 1
Enter the element to push: 2
2 inserted into the stack
----- STACK MENU -----
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 2
2 deleted from the stack
----- STACK MENU -----
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 3
Stack is empty
----- STACK MENU -----
1. Push
2. Pop
3. Display
4. Exit
Enter your choice: 4
Program terminated.
>>>
Ln: 39 Col: 0

# Week-8

8a) Linear Queue:

## CODE:



```
2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help

class LinearQueue:
    def __init__(self, size):
        self.size = size
        self.queue = [None] * size
        self.front = -1
        self.rear = -1

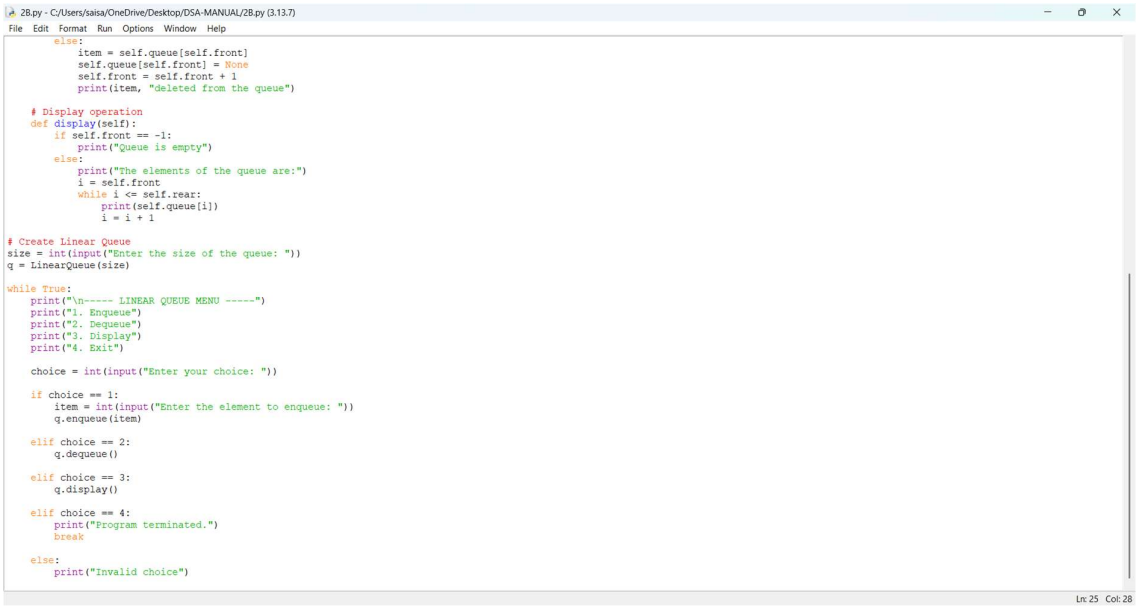
    # Enqueue operation
    def enqueue(self, item):
        if self.rear == self.size - 1:
            print("Queue Overflow")
        elif self.front == -1:
            self.front = 0
            self.rear = 0
            self.queue[self.rear] = item
            print(item, "inserted into the queue")
        else:
            self.rear = self.rear + 1
            self.queue[self.rear] = item
            print(item, "inserted into the queue")

    # Dequeue operation
    def dequeue(self):
        if self.front == -1:
            print("Queue Underflow")

        # Only one element is present
        elif self.front == self.rear:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = -1
            self.rear = -1
            print(item, "deleted from the queue")

        # More than one element
        else:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = self.front + 1
            print(item, "deleted from the queue")

    # Display operation
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
Ln: 83  Col: 0
```



```
2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help
        else:
            item = self.queue[self.front]
            self.queue[self.front] = None
            self.front = self.front + 1
            print(item, "deleted from the queue")

    # Display operation
    def display(self):
        if self.front == -1:
            print("Queue is empty")
        else:
            print("The elements of the queue are:")
            i = self.front
            while i <= self.rear:
                print(self.queue[i])
                i = i + 1

# Create Linear Queue
size = int(input("Enter the size of the queue: "))
q = LinearQueue(size)

while True:
    print("\n----- LINEAR QUEUE MENU -----")
    print("1. Enqueue")
    print("2. Dequeue")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        item = int(input("Enter the element to enqueue: "))
        q.enqueue(item)

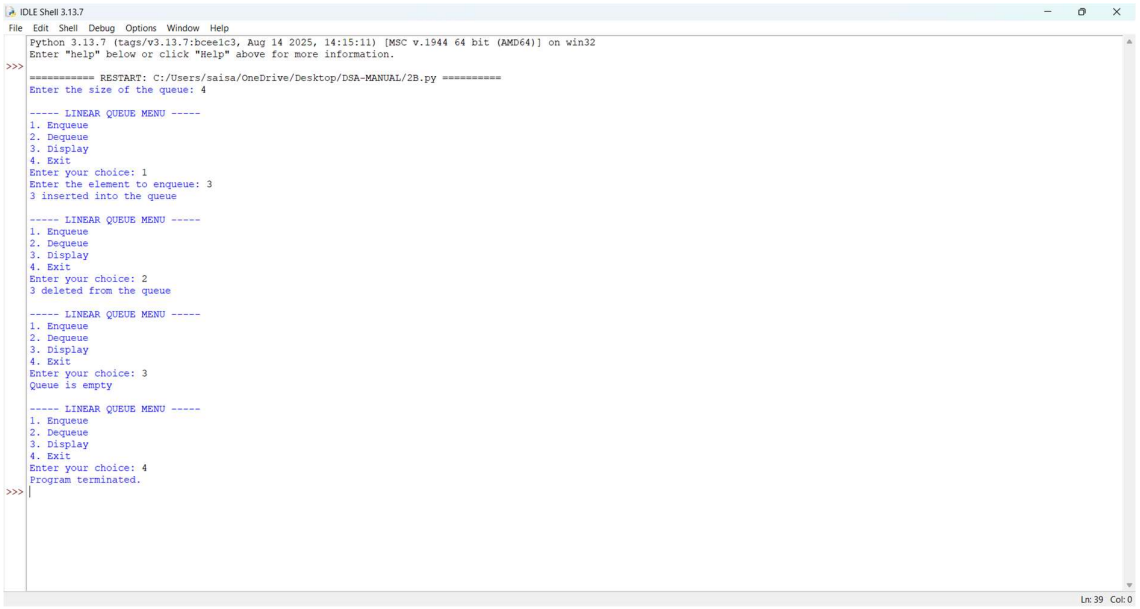
    elif choice == 2:
        q.dequeue()

    elif choice == 3:
        q.display()

    elif choice == 4:
        print("Program terminated.")
        break

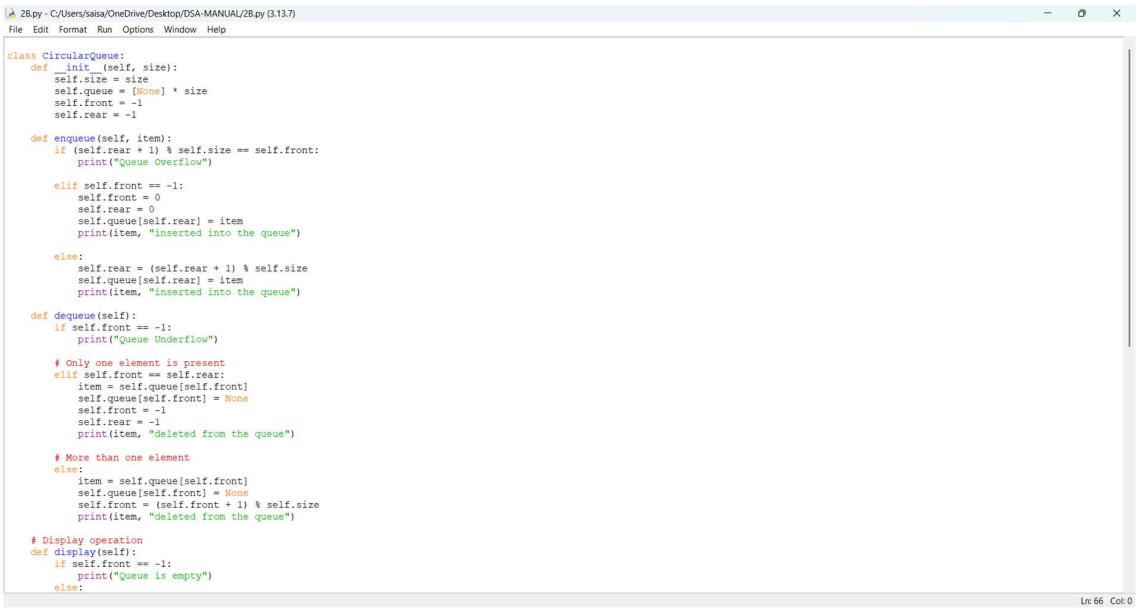
    else:
        print("Invalid choice")
Ln: 25  Col: 28
```

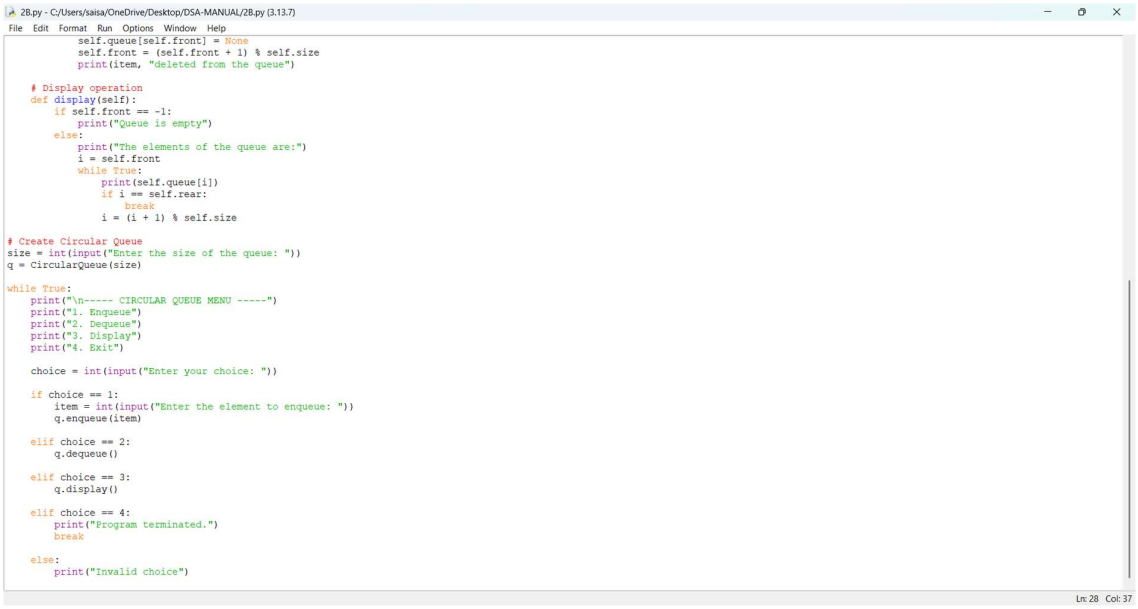
## OUTPUT:




8b) Circular queue:

## CODE:




2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)
File Edit Format Run Options Window Help
self.queue[self.front] = None
self.front = (self.front + 1) % self.size
print(item, "deleted from the queue")
# Display operation
def display(self):
if self.front == -1:
print("Queue is empty")
else:
print("The elements of the queue are:")
i = self.front
while True:
print(self.queue[i])
if i == self.rear:
break
i = (i + 1) % self.size
# Create Circular Queue
size = int(input("Enter the size of the queue: "))
q = CircularQueue(size)
while True:
print("\n----- CIRCULAR QUEUE MENU -----")
print("1. Enqueue")
print("2. Dequeue")
print("3. Display")
print("4. Exit")
choice = int(input("Enter your choice: "))
if choice == 1:
item = int(input("Enter the element to enqueue: "))
q.enqueue(item)
elif choice == 2:
q.dequeue()
elif choice == 3:
q.display()
elif choice == 4:
print("Program terminated.")
break
else:
print("Invalid choice")
Ln: 28 Col: 37

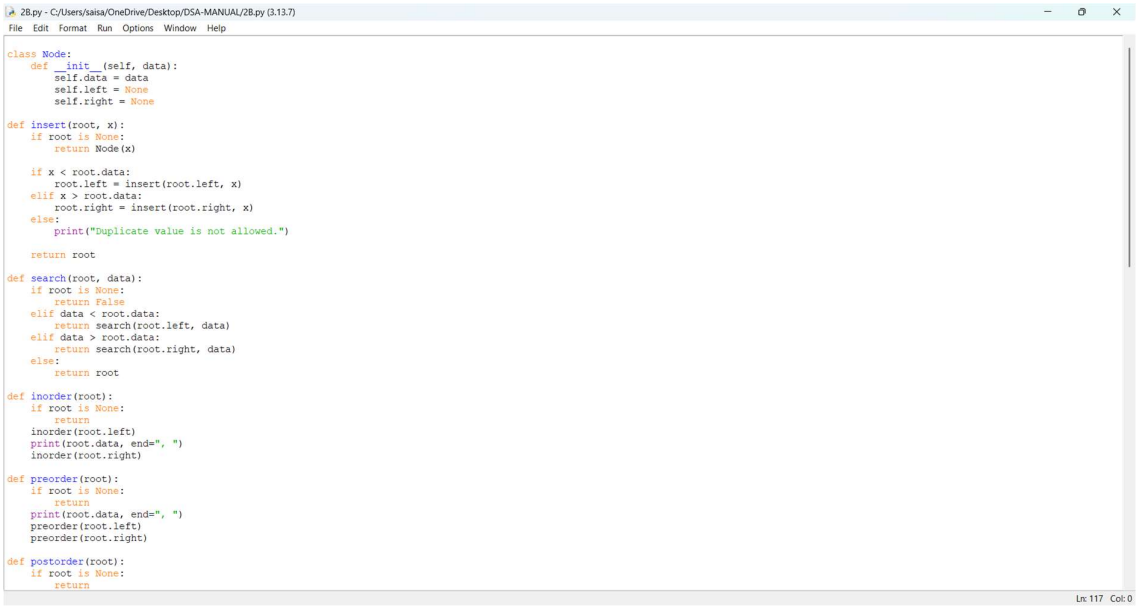
## OUTPUT:

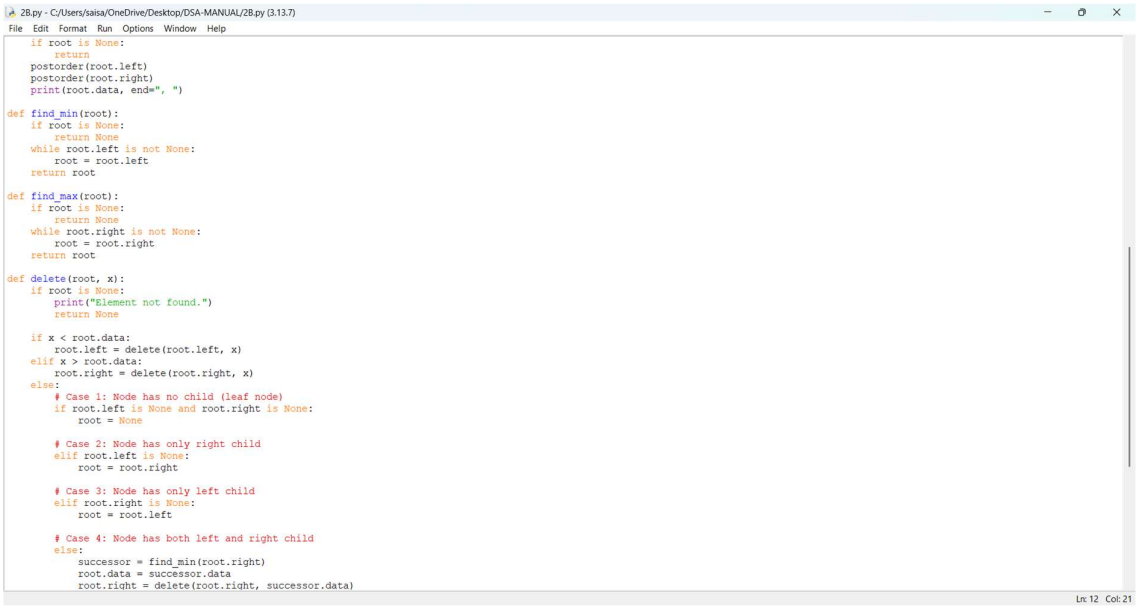

IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py ==========
Enter the size of the queue: 4
----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 1
Enter the element to enqueue: 3
3 inserted into the queue
----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 2
3 deleted from the queue
----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 3
Queue is empty
----- CIRCULAR QUEUE MENU -----
1. Enqueue
2. Dequeue
3. Display
4. Exit
Enter your choice: 4
Program terminated.
>>>
Ln: 39 Col: 0

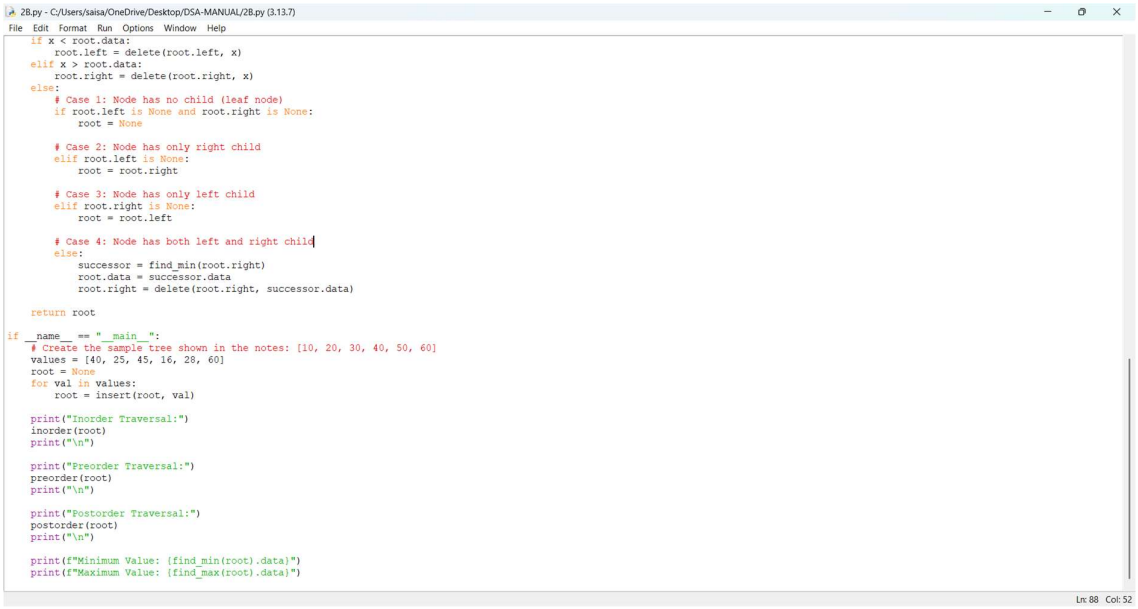
# Week-9

9)BST TREE OPERATIONS:

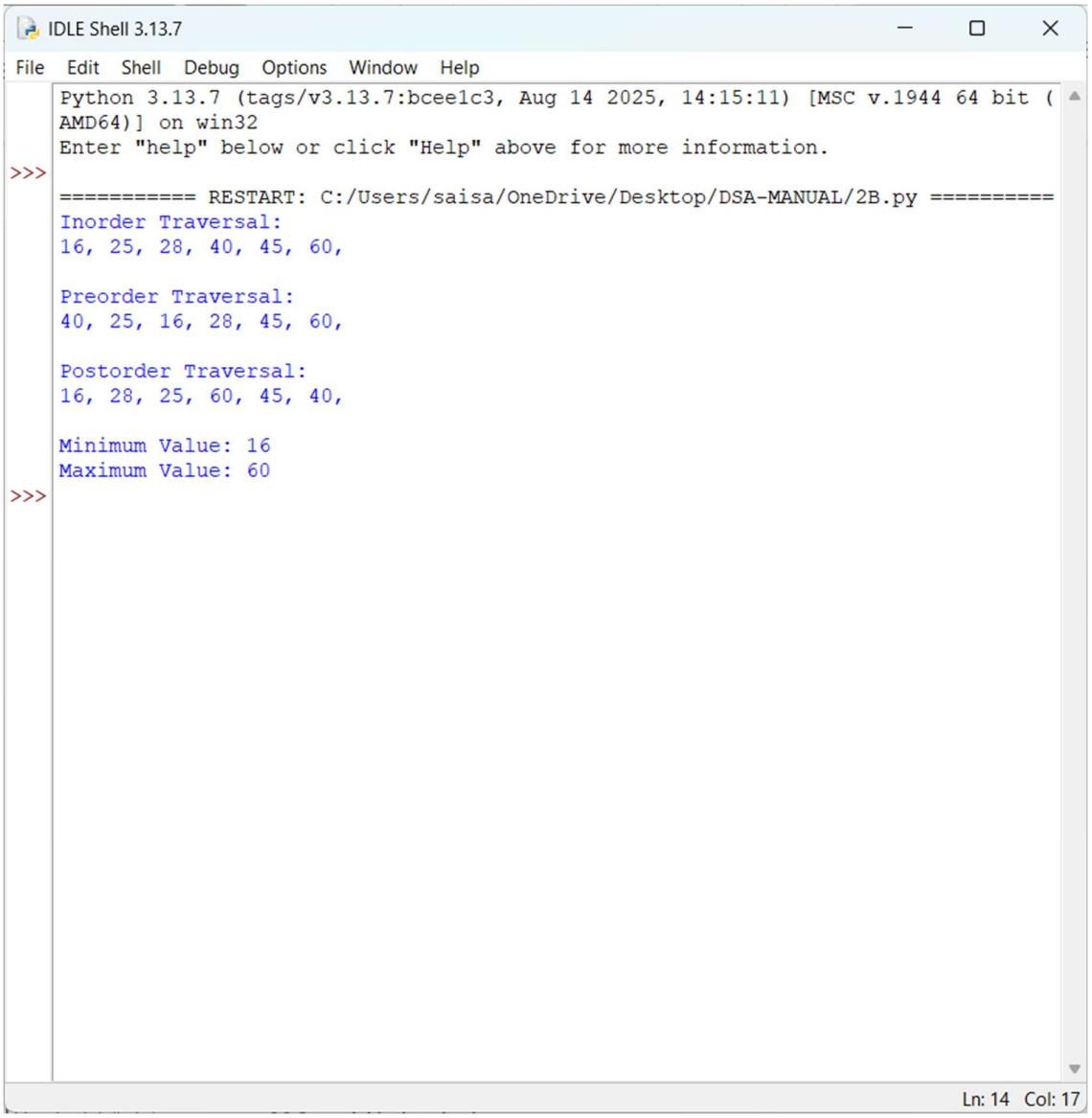
## CODE:




2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)
File Edit Format Run Options Window Help
if x < root.data:
root.left = delete(root.left, x)
elif x > root.data:
root.right = delete(root.right, x)
else:
# Case 1: Node has no child (leaf node)
if root.left is None and root.right is None:
root = None
# Case 2: Node has only right child
elif root.left is None:
root = root.right
# Case 3: Node has only left child
elif root.right is None:
root = root.left
# Case 4: Node has both left and right child
else:
successor = find_min(root.right)
root.data = successor.data
root.right = delete(root.right, successor.data)
return root
if __name__ == "__main__":
# Create the sample tree shown in the notes: [10, 20, 30, 40, 50, 60]
values = [40, 25, 45, 16, 28, 60]
root = None
for val in values:
root = insert(root, val)
print("Inorder Traversal:")
inorder(root)
print("\n")
print("Preorder Traversal:")
preorder(root)
print("\n")
print("Postorder Traversal:")
postorder(root)
print("\n")
print(f"Minimum Value: {find_min(root).data}")
print(f"Maximum Value: {find_max(root).data}")
Ln: 88 Col: 52

## OUTPUT:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py ==========
Inorder Traversal:
16, 25, 28, 40, 45, 60,
Preorder Traversal:
40, 25, 16, 28, 45, 60,
Postorder Traversal:
16, 28, 25, 60, 45, 40,
Minimum Value: 16
Maximum Value: 60
>>>
Ln: 14 Col: 17

# Week-10

10)AVL TREE OPERATIONS:

## CODE:



```
2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help

class Node:
    def __init__(self, x):
        self.left = None
        self.data = x
        self.right = None
        self.h = 0 # Height of the node

def height(root):
    if root is None:
        return -1
    else:
        return root.h

def LL(x):
    y = x.left
    x.left = y.right
    y.right = x

    # Update heights
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

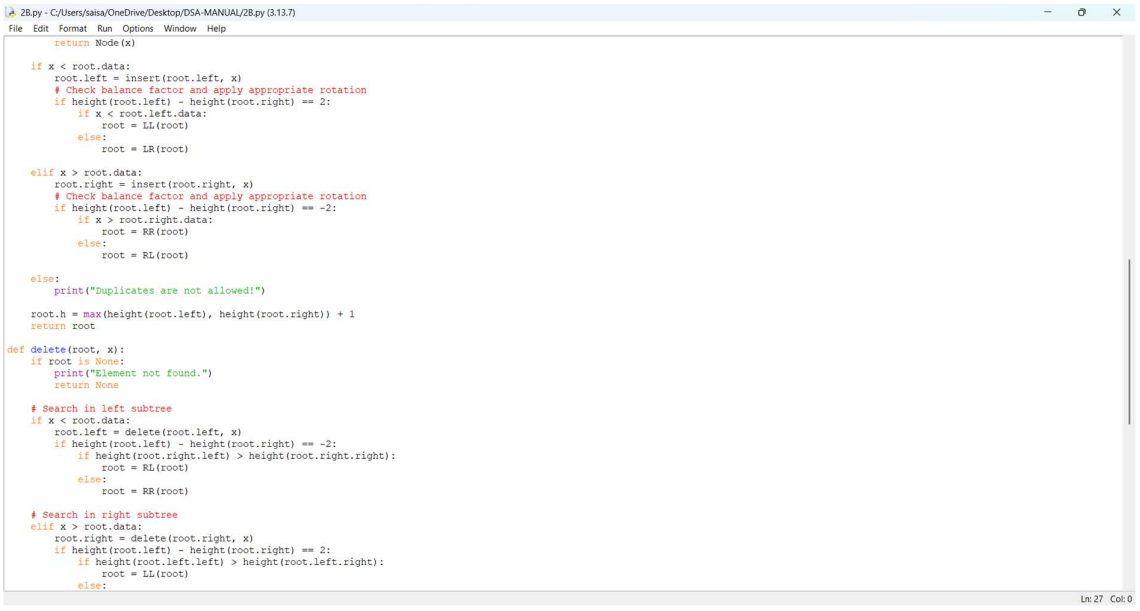
def RR(x):
    y = x.right
    x.right = y.left
    y.left = x

    # Update heights
    x.h = max(height(x.left), height(x.right)) + 1
    y.h = max(height(y.left), height(y.right)) + 1
    return y

def LR(x):
    x.left = RR(x.left)
    return LL(x)

def RL(x):
    x.right = LL(x.right)
    return RR(x)

def inorder(root):
    if root is None:
        return
    inorder(root.left)
    print(root.data, end=", ")
Ln: 139  Col: 0
```



```
2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help
        return Node(x)

    if x < root.data:
        root.left = insert(root.left, x)
        # Check balance factor and apply appropriate rotation
        if height(root.left) - height(root.right) == 2:
            if x < root.left.data:
                root = LL(root)
            else:
                root = LR(root)

    elif x > root.data:
        root.right = insert(root.right, x)
        # Check balance factor and apply appropriate rotation
        if height(root.left) - height(root.right) == -2:
            if x > root.right.data:
                root = RR(root)
            else:
                root = RL(root)

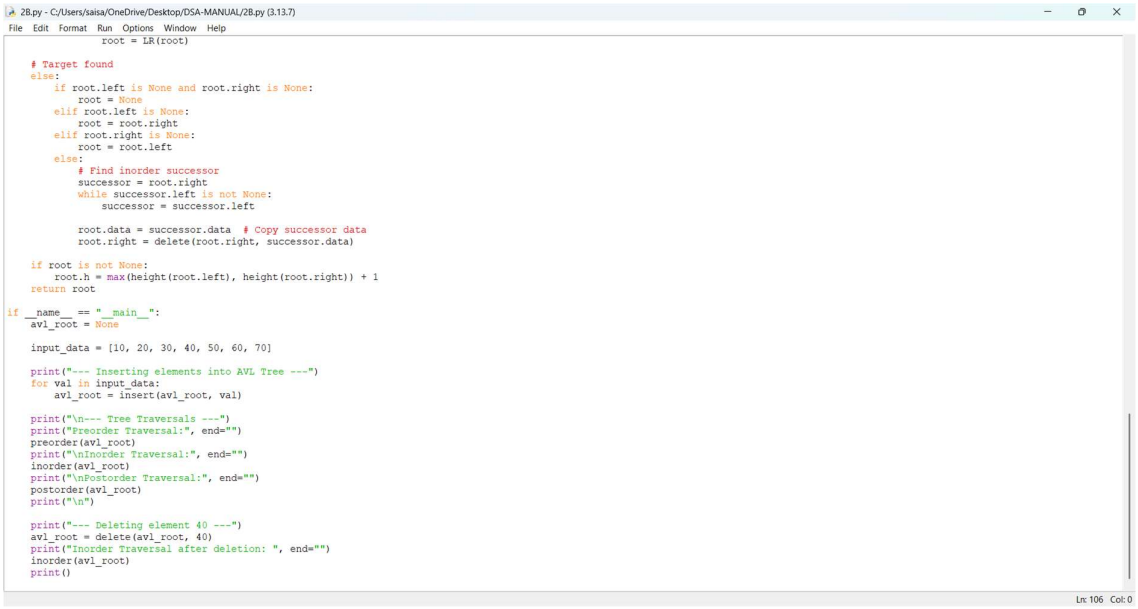
    else:
        print("Duplicates are not allowed!")

    root.h = max(height(root.left), height(root.right)) + 1
    return root

def delete(root, x):
    if root is None:
        print("Element not found.")
        return None

    # Search in left subtree
    if x < root.data:
        root.left = delete(root.left, x)
        if height(root.left) - height(root.right) == -2:
            if height(root.right.left) > height(root.right.right):
                root = RL(root)
            else:
                root = RR(root)

    # Search in right subtree
    elif x > root.data:
        root.right = delete(root.right, x)
        if height(root.left) - height(root.right) == 2:
            if height(root.left.left) > height(root.left.right):
                root = LL(root)
            else:
Ln: 27  Col: 0
```



```
2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help
                root = LR(root)

    # Target found
    else:
        if root.left is None and root.right is None:
            root = None
        elif root.left is None:
            root = root.right
        elif root.right is None:
            root = root.left
        else:
            # Find inorder successor
            successor = root.right
            while successor.left is not None:
                successor = successor.left

            root.data = successor.data  # Copy successor data
            root.right = delete(root.right, successor.data)

    if root is not None:
        root.h = max(height(root.left), height(root.right)) + 1
    return root

if __name__ == "__main__":
    avl_root = None

    input_data = [10, 20, 30, 40, 50, 60, 70]

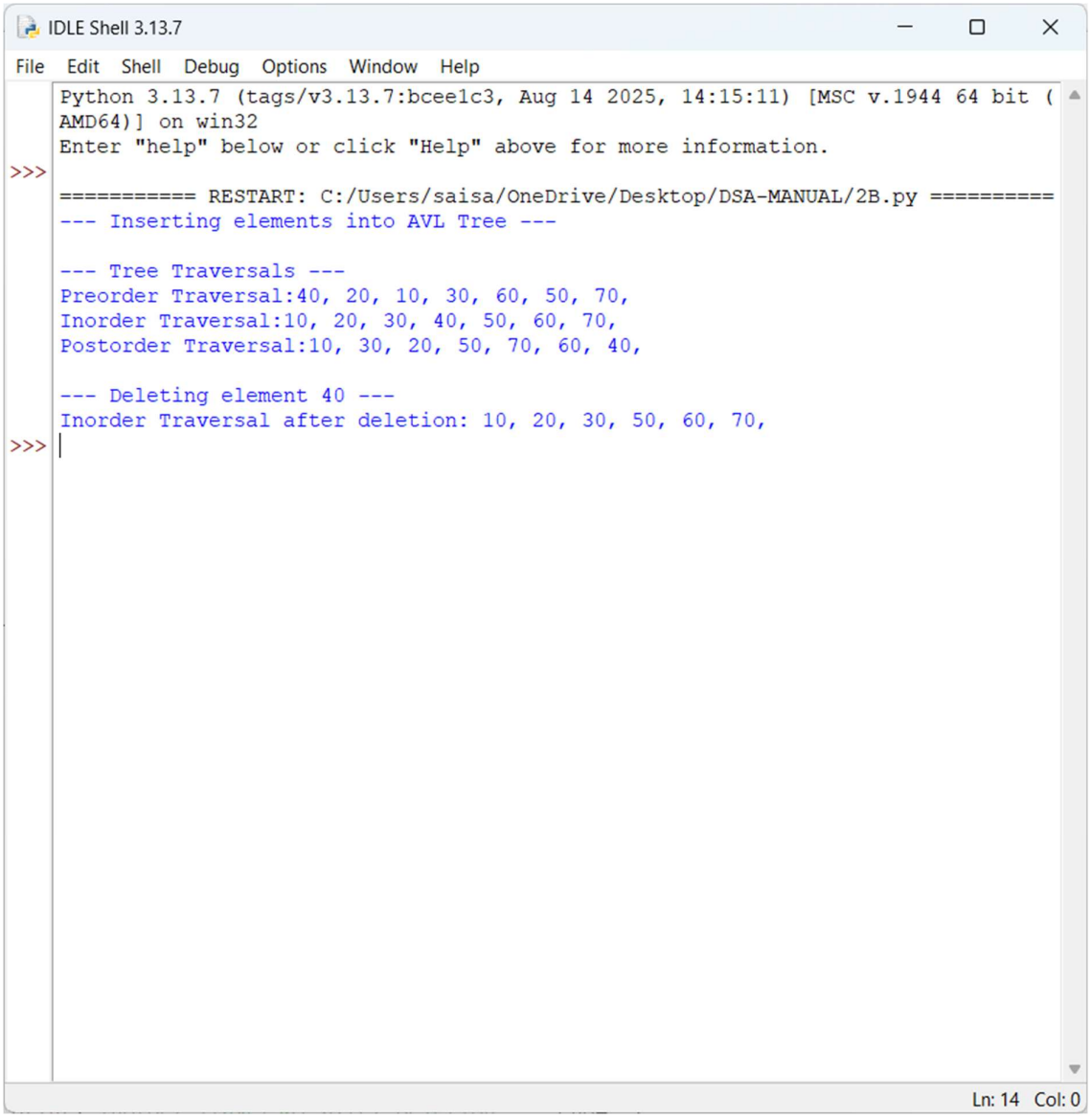
    print("--- Inserting elements into AVL Tree ---")
    for val in input_data:
        avl_root = insert(avl_root, val)

    print("\n--- Tree Traversals ---")
    print("Preorder Traversal:", end="")
    preorder(avl_root)
    print("\nInorder Traversal:", end="")
    inorder(avl_root)
    print("\nPostorder Traversal:", end="")
    postorder(avl_root)
    print("\n")

    print("--- Deleting element 40 ---")
    avl_root = delete(avl_root, 40)
    print("Inorder Traversal after deletion: ", end="")
    inorder(avl_root)
    print()
```

Ln: 106  Col: 0

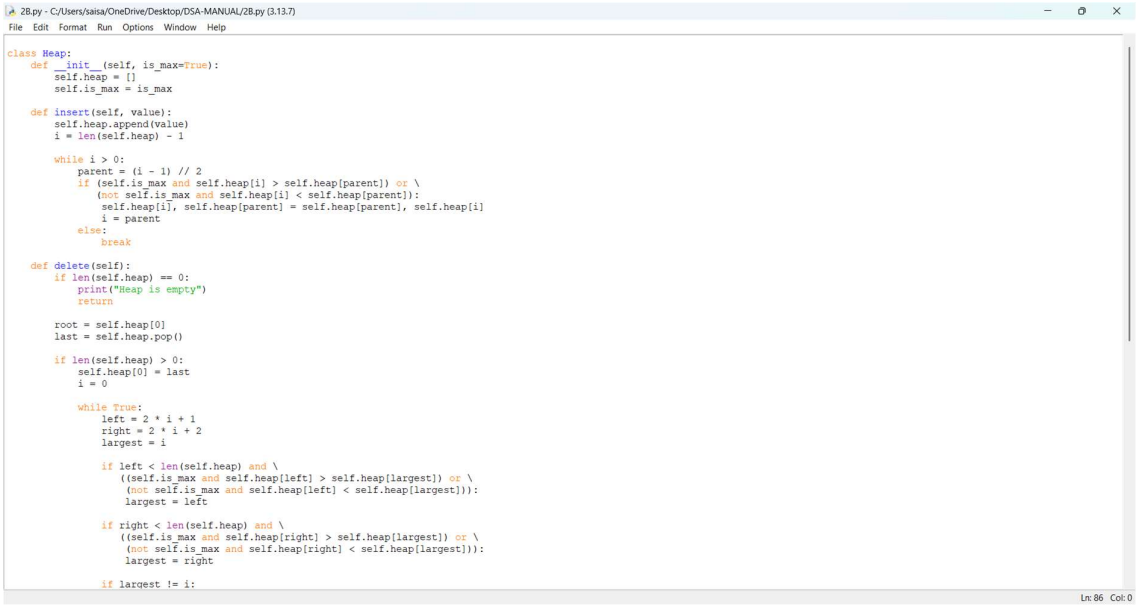
## OUTPUT:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py ==========
--- Inserting elements into AVL Tree ---
--- Tree Traversals ---
Preorder Traversal:40, 20, 10, 30, 60, 50, 70,
Inorder Traversal:10, 20, 30, 40, 50, 60, 70,
Postorder Traversal:10, 30, 20, 50, 70, 60, 40,
--- Deleting element 40 ---
Inorder Traversal after deletion: 10, 20, 30, 50, 60, 70,
>>>
Ln: 14 Col: 0

# Week-11

11)HEAP TREE:

## CODE:



```
2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help

class Heap:
    def __init__(self, is_max=True):
        self.heap = []
        self.is_max = is_max

    def insert(self, value):
        self.heap.append(value)
        i = len(self.heap) - 1

        while i > 0:
            parent = (i - 1) // 2
            if (self.is_max and self.heap[i] > self.heap[parent]) or \
               (not self.is_max and self.heap[i] < self.heap[parent]):
                self.heap[i], self.heap[parent] = self.heap[parent], self.heap[i]
                i = parent
            else:
                break

    def delete(self):
        if len(self.heap) == 0:
            print("Heap is empty")
            return

        root = self.heap[0]
        last = self.heap.pop()

        if len(self.heap) > 0:
            self.heap[0] = last
            i = 0

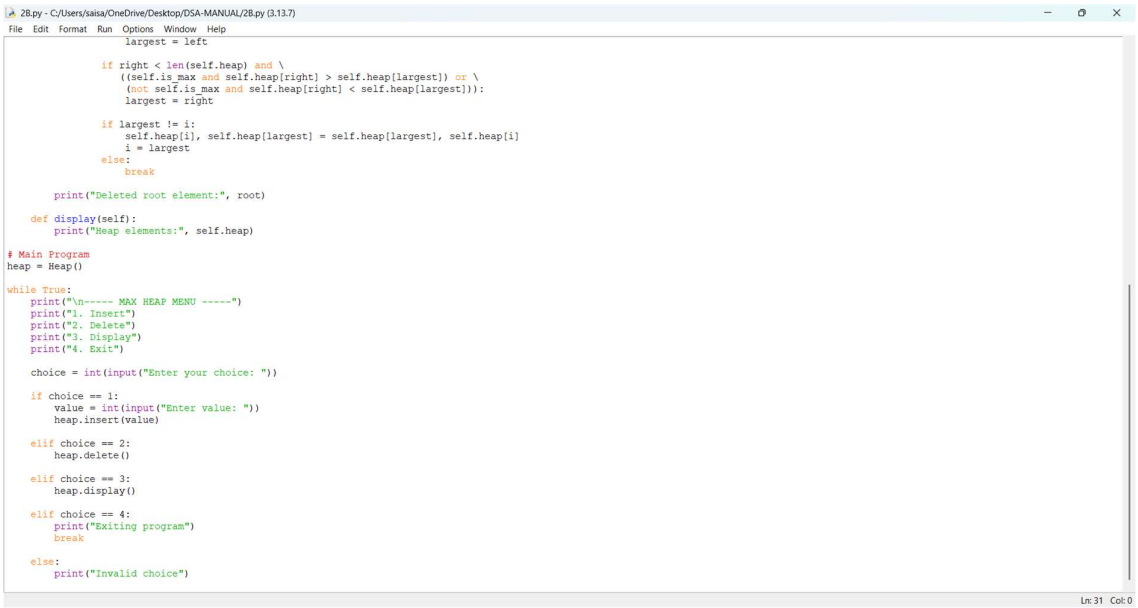
            while True:
                left = 2 * i + 1
                right = 2 * i + 2
                largest = i

                if left < len(self.heap) and \
                   ((self.is_max and self.heap[left] > self.heap[largest]) or \
                    (not self.is_max and self.heap[left] < self.heap[largest])):
                    largest = left

                if right < len(self.heap) and \
                   ((self.is_max and self.heap[right] > self.heap[largest]) or \
                    (not self.is_max and self.heap[right] < self.heap[largest])):
                    largest = right

                if largest != i:

Ln: 86  Col: 0
```



```
2B.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help
                    largest = left

                if right < len(self.heap) and \
                   ((self.is_max and self.heap[right] > self.heap[largest]) or \
                    (not self.is_max and self.heap[right] < self.heap[largest])):
                    largest = right

                if largest != i:
                    self.heap[i], self.heap[largest] = self.heap[largest], self.heap[i]
                    i = largest
                else:
                    break

        print("Deleted root element:", root)

    def display(self):
        print("Heap elements:", self.heap)

# Main Program
heap = Heap()

while True:
    print("\n----- MAX HEAP MENU -----")
    print("1. Insert")
    print("2. Delete")
    print("3. Display")
    print("4. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        value = int(input("Enter value: "))
        heap.insert(value)

    elif choice == 2:
        heap.delete()

    elif choice == 3:
        heap.display()

    elif choice == 4:
        print("Exiting program")
        break

    else:
        print("Invalid choice")

Ln: 31  Col: 0
```

## OUTPUT:

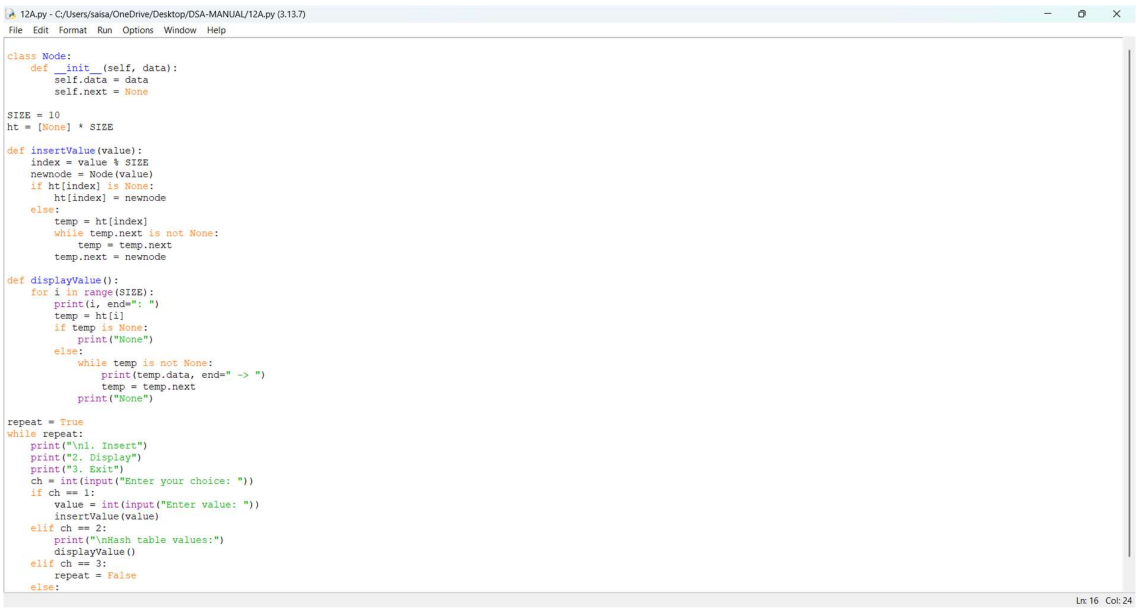

IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/2B.py ==========
----- MAX HEAP MENU -----
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 1
Enter value: 2
----- MAX HEAP MENU -----
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 2
Deleted root element: 2
----- MAX HEAP MENU -----
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 3
Heap elements: []
----- MAX HEAP MENU -----
1. Insert
2. Delete
3. Display
4. Exit
Enter your choice: 4
Exiting program
>>>
Ln: 37 Col: 0

# Week-12

12)HASHING:

a) SEPERATE CHAINING:

## CODE:



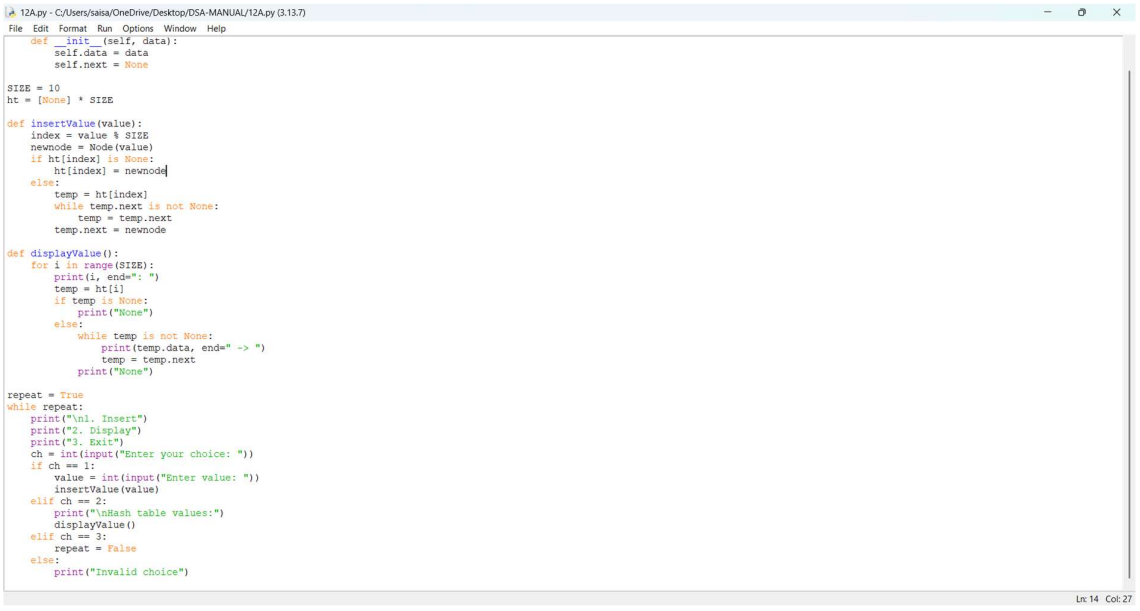
```
12A.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/12A.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

SIZE = 10
ht = [None] * SIZE

def insertValue(value):
    index = value % SIZE
    newnode = Node(value)
    if ht[index] is None:
        ht[index] = newnode
    else:
        temp = ht[index]
        while temp.next is not None:
            temp = temp.next
        temp.next = newnode

def displayValue():
    for i in range(SIZE):
        print(i, end=": ")
        temp = ht[i]
        if temp is None:
            print("None")
        else:
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("\nHash table values:")
        displayValue()
    elif ch == 3:
        repeat = False
    else:
Ln: 16  Col: 24
```



```
12A.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/12A.py (3.13.7)
File  Edit  Format  Run  Options  Window  Help
    def __init__(self, data):
        self.data = data
        self.next = None

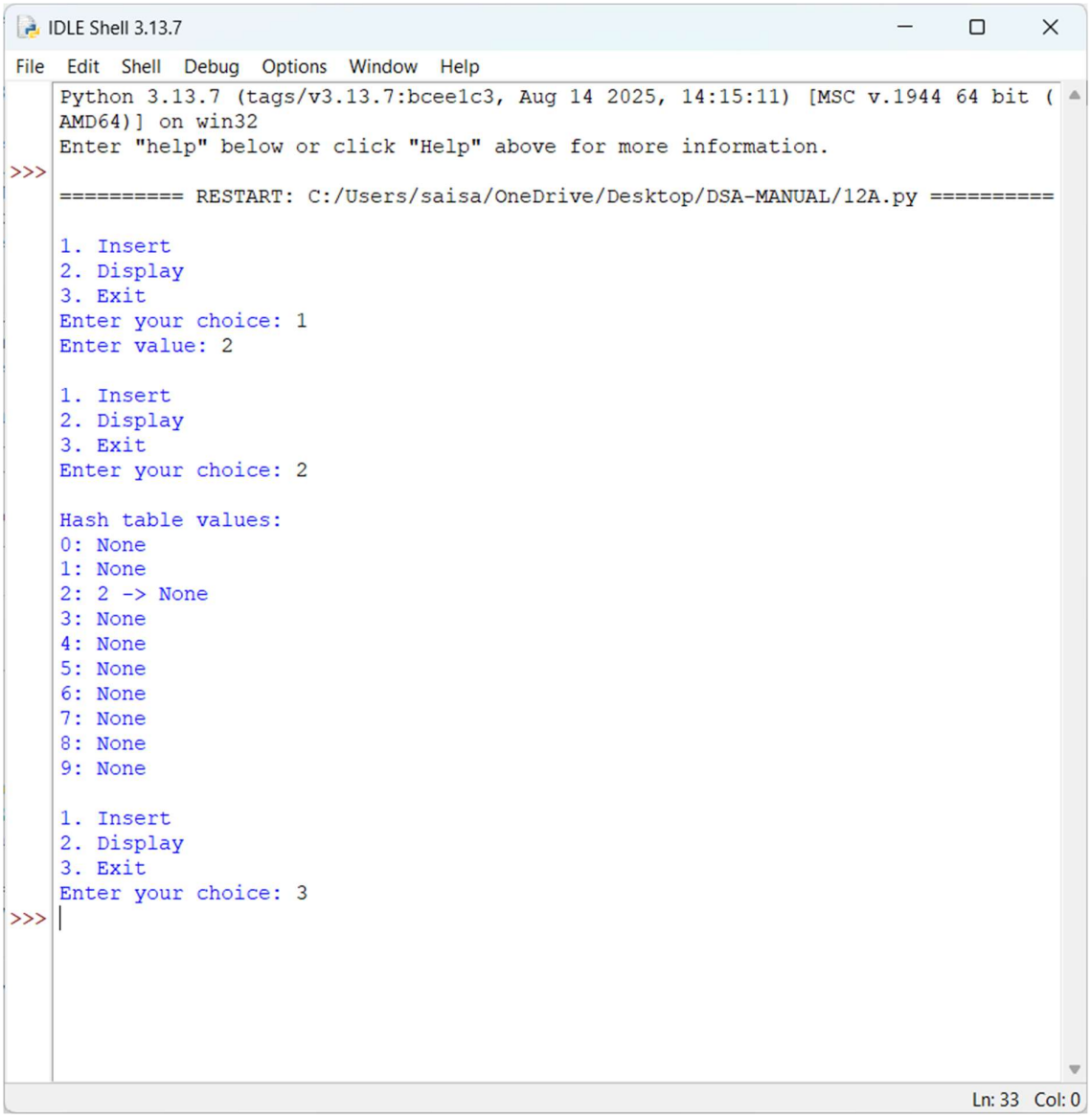
SIZE = 10
ht = [None] * SIZE

def insertValue(value):
    index = value % SIZE
    newnode = Node(value)
    if ht[index] is None:
        ht[index] = newnode
    else:
        temp = ht[index]
        while temp.next is not None:
            temp = temp.next
        temp.next = newnode

def displayValue():
    for i in range(SIZE):
        print(i, end=": ")
        temp = ht[i]
        if temp is None:
            print("None")
        else:
            while temp is not None:
                print(temp.data, end=" -> ")
                temp = temp.next
            print("None")

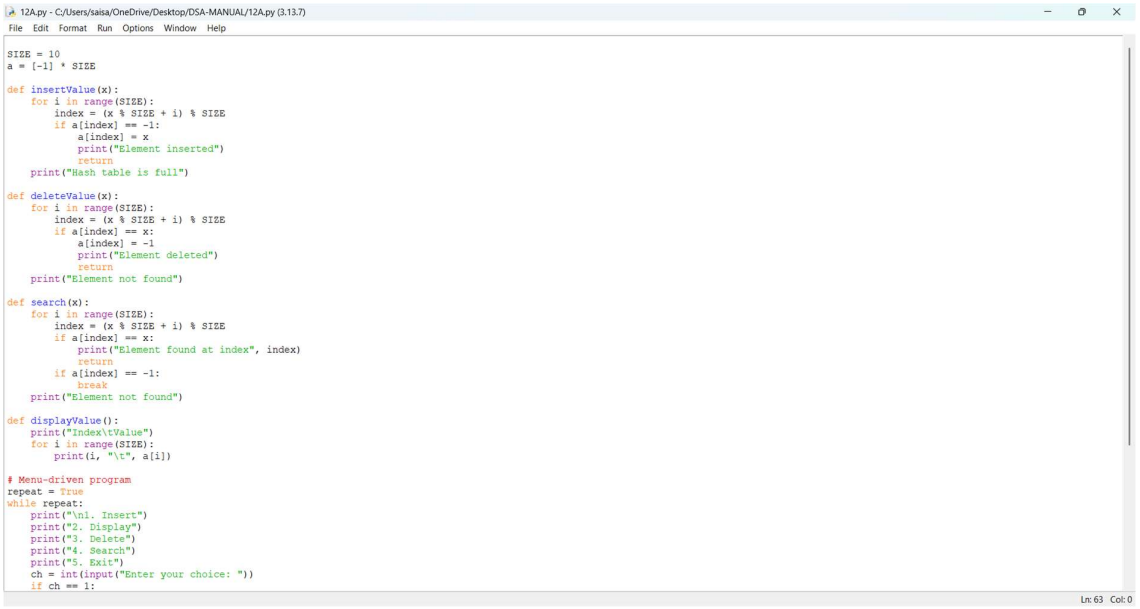
repeat = True
while repeat:
    print("\n1. Insert")
    print("2. Display")
    print("3. Exit")
    ch = int(input("Enter your choice: "))
    if ch == 1:
        value = int(input("Enter value: "))
        insertValue(value)
    elif ch == 2:
        print("\nHash table values:")
        displayValue()
    elif ch == 3:
        repeat = False
    else:
        print("Invalid choice")
Ln: 14  Col: 27
```

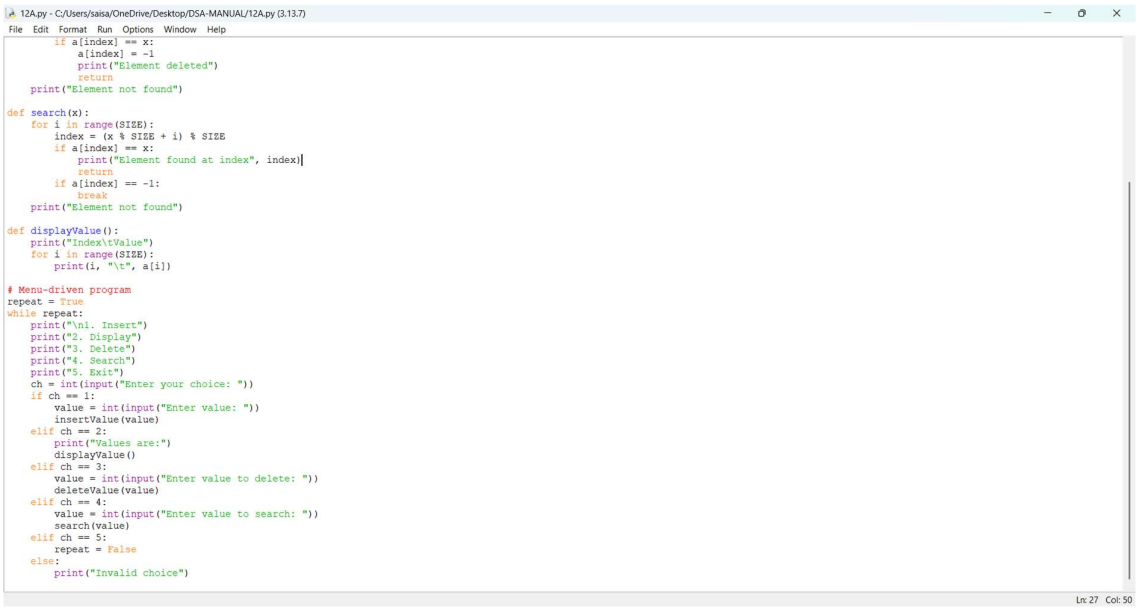
## OUTPUT:



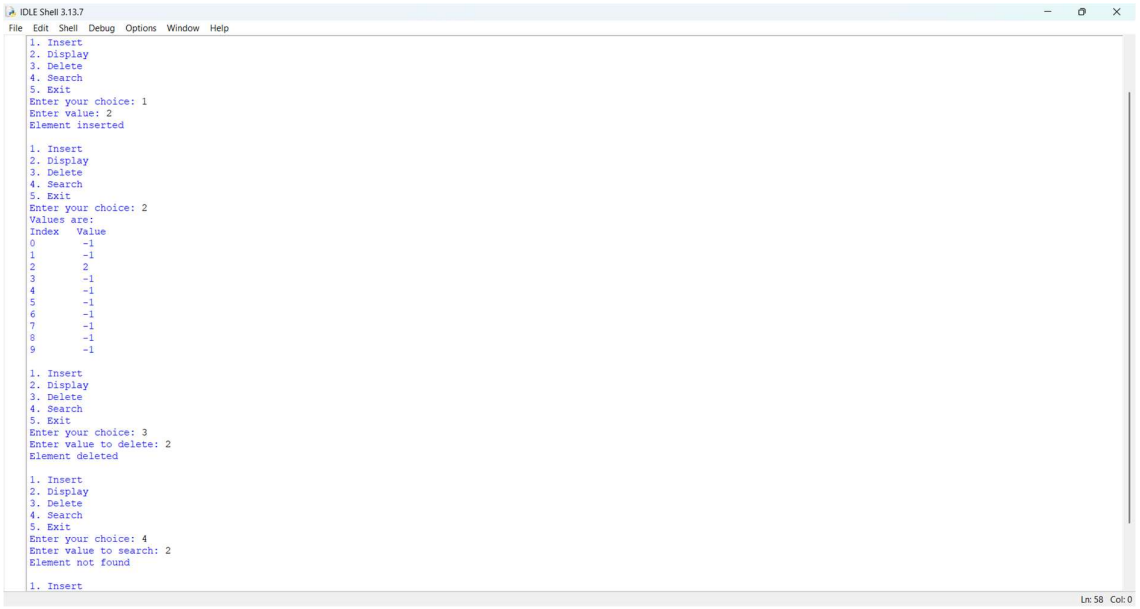

b) LINEAR PROBING

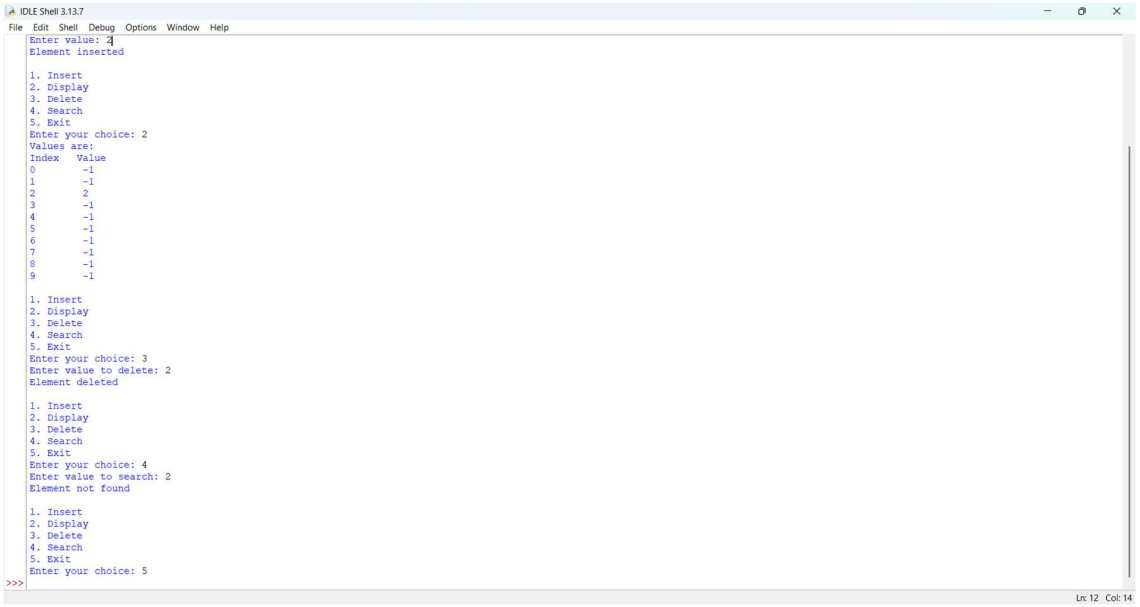
**CODE:**


12A.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/12A.py (3.13.7)
File Edit Format Run Options Window Help
SIZE = 10
a = [-1] * SIZE
def insertValue(x):
for i in range(SIZE):
index = (x % SIZE + i) % SIZE
if a[index] == -1:
a[index] = x
print("Element inserted")
return
print("Hash table is full")
def deleteValue(x):
for i in range(SIZE):
index = (x % SIZE + i) % SIZE
if a[index] == x:
a[index] = -1
print("Element deleted")
return
print("Element not found")
def search(x):
for i in range(SIZE):
index = (x % SIZE + i) % SIZE
if a[index] == x:
print("Element found at index", index)
return
if a[index] == -1:
break
print("Element not found")
def displayValue():
print("Index\tValue")
for i in range(SIZE):
print(i, "\t", a[i])
# Menu-driven program
repeat = True
while repeat:
print("\n1. Insert")
print("2. Display")
print("3. Delete")
print("4. Search")
print("5. Exit")
ch = int(input("Enter your choice: "))
if ch == 1:
Ln: 63 Col: 0


12A.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/12A.py (3.13.7)
File Edit Format Run Options Window Help
if a[index] == x:
a[index] = -1
print("Element deleted")
return
print("Element not found")
def search(x):
for i in range(SIZE):
index = (x % SIZE + i) % SIZE
if a[index] == x:
print("Element found at index", index)
return
if a[index] == -1:
break
print("Element not found")
def displayValue():
print("Index\tValue")
for i in range(SIZE):
print(i, "\t", a[i])
# Menu-driven program
repeat = True
while repeat:
print("\n1. Insert")
print("2. Display")
print("3. Delete")
print("4. Search")
print("5. Exit")
ch = int(input("Enter your choice: "))
if ch == 1:
value = int(input("Enter value: "))
insertValue(value)
elif ch == 2:
print("Values are:")
displayValue()
elif ch == 3:
value = int(input("Enter value to delete: "))
deleteValue(value)
elif ch == 4:
value = int(input("Enter value to search: "))
search(value)
elif ch == 5:
repeat = False
else:
print("Invalid choice")
Ln: 27 Col: 50

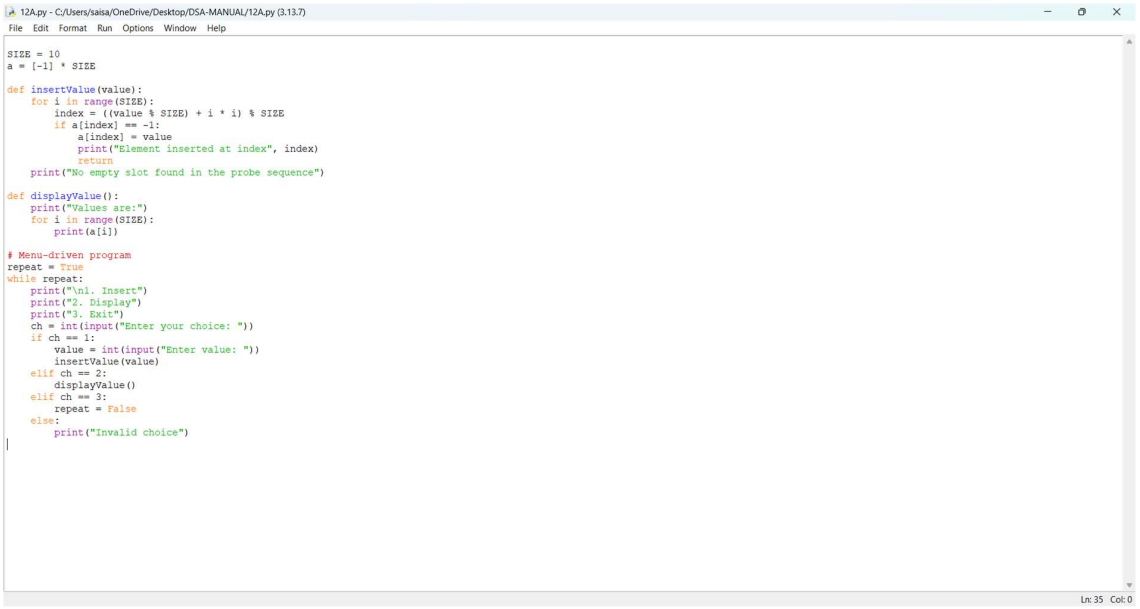
# OUTPUT:



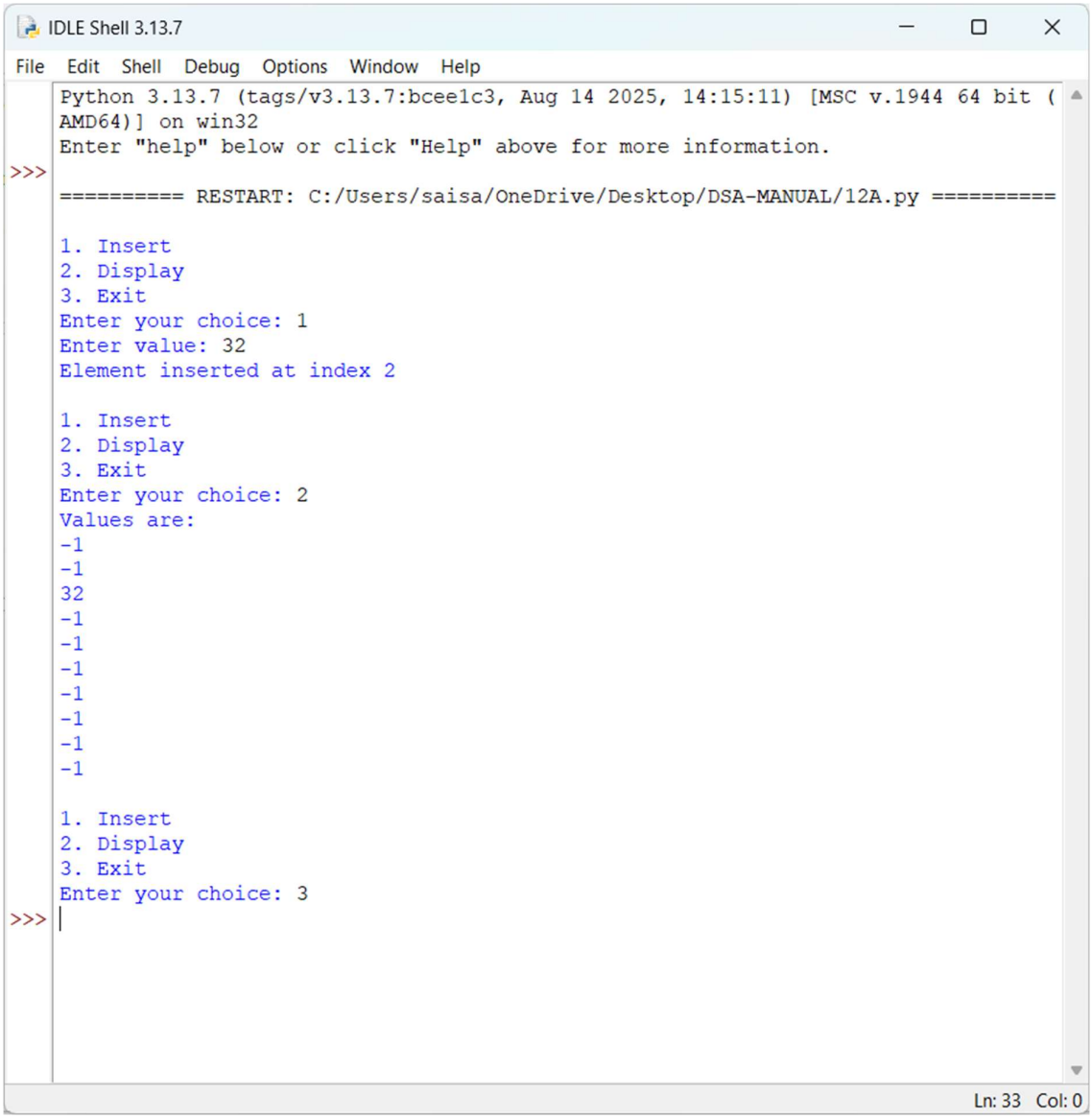




c) HASHING IN QUADRATIC PROBING:

**CODE:**


12A.py - C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/12A.py (3.13.7)
File Edit Format Run Options Window Help
SIZE = 10
a = [-1] * SIZE
def insertValue(value):
for i in range(SIZE):
index = ((value % SIZE) + i * i) % SIZE
if a[index] == -1:
a[index] = value
print("Element inserted at index", index)
return
print("No empty slot found in the probe sequence")
def displayValue():
print("Values are:")
for i in range(SIZE):
print(a[i])
# Menu-driven program
repeat = True
while repeat:
print("\n1. Insert")
print("2. Display")
print("3. Exit")
ch = int(input("Enter your choice: "))
if ch == 1:
value = int(input("Enter value: "))
insertValue(value)
elif ch == 2:
displayValue()
elif ch == 3:
repeat = False
else:
print("Invalid choice")
Ln: 35 Col: 0

## OUTPUT:


IDLE Shell 3.13.7
File Edit Shell Debug Options Window Help
Python 3.13.7 (tags/v3.13.7:bcee1c3, Aug 14 2025, 14:15:11) [MSC v.1944 64 bit (
AMD64)] on win32
Enter "help" below or click "Help" above for more information.
>>>
========== RESTART: C:/Users/saisa/OneDrive/Desktop/DSA-MANUAL/12A.py ==========
1. Insert
2. Display
3. Exit
Enter your choice: 1
Enter value: 32
Element inserted at index 2
1. Insert
2. Display
3. Exit
Enter your choice: 2
Values are:
-1
-1
32
-1
-1
-1
-1
-1
-1
1. Insert
2. Display
3. Exit
Enter your choice: 3
>>>
Ln: 33 Col: 0